CE-LSLM: Efficient Large-Small Language Model Inference and Communication via Cloud-Edge Collaboration

Pengyan Zhu, Tingting Yang*

***Abstract*—Emerging intelligent service scenarios in 6G communication impose stringent requirements for low latency, high reliability, and privacy preservation. Generative large language models (LLMs) are gradually becoming key enablers for the integration of semantic communication and computation. However, due to the limited computational resources of edge devices and the increasing complexity of heterogeneous terminal access, existing centralized inference approaches fail to meet the dual demands of response efficiency and data privacy in edge-side inference tasks. To address these challenges, this paper proposes a novel collaborative inference architecture that integrates cloud-based LLMs with edge-deployed small language models (SLMs), enabling dynamic scheduling and sharing of semantic-level intermediate states, and establishing a unified computation-communication paradigm tailored for 6G networks. Specifically, a key-value (KV) cache reuse mechanism is introduced to enhance the semantic understanding of edge models through contextual guidance from the cloud, while significantly reducing edge-side computational and storage overhead. Furthermore, a cross-node parallel scheduling mechanism is proposed to achieve asynchronous coordination between model state loading and decoding computation, thereby improving edge responsiveness. In addition, we investigate layer alignment and representation compression strategies between heterogeneous models to alleviate the communication burden on the edge. Experimental results demonstrate that the proposed architecture exhibits superior adaptability and scalability in terms of inference latency, system stability, and concurrent processing capacity.**

***Index Terms*—Large language model inference; 6G communications; Heterogeneous model collaboration; Privacy preservation; Resource-constrained edge devices**

I. Introduction

WITH the large-scale commercial deployment of fifth-generation (5G) cellular systems, there is a renewed desire to explore new solutions to solve the real-world problems of relatively small coverage, spectrum resource constraints, and poor device compatibility to improve the existing lifestyles. As the successor to 5G communication technology, sixth-generation (6G) will not just be a wireless network with higher transmission rates and greater network capacity, but a digitally-connected, fully automated and intelligent global communication network capable of delivering diverse network services for different scenarios, including industrial intelligence, smart wearables, autonomous vehicles, and smart healthcare system [1]. Terahertz communication, a cornerstone technology for 6G, leverages electromagnetic waves within the 0.1 THz to 10 THz frequency range to achieve data transmission rates of up to 1 Tbps. This capability supports the high bandwidth and ultra-low latency demands of specialized 6G task scenarios [2], [3], [4]. However, due to the short wavelength of terahertz waves, combined with their limited penetration and propagation distance, adequate signal coverage requires the deployment of dense small-scale models or edge nodes to establish multi-tier network architectures, encompassing both near-edge and far-edge distributed scenarios. From a practical perspective, while multi-tier network architectures can mitigate signal coverage issues, edge nodes face significant challenges in directly handling complex inference tasks due to constraints in computational power, memory, and bandwidth resources. Furthermore, with the exponential growth in device connectivity, 6G must address not only the insufficiency of computational and transmission resources but also the critical need for robust user privacy protection.

The emergence of large language models (LLMs), exemplified by ChatGPT, will provide a significant driving force and key support for the development of 6G. Models such as the GPT series and BERT, with billions or even hundreds of billions of parameters, possess high-precision natural language understanding and generation capabilities. They can extract more features and knowledge from vast amounts of data, offering greater potential for solving complex real-world problems. However, due to the immense size of these models, storing and processing their parameters requires substantial memory. For resource-constrained devices, the widespread deployment of LLMs is challenging.

Existing research has improved the training and inference efficiency of LLMs to some extent by restructuring GPU inference cluster architectures and employing advanced hardware-software co-optimization techniquesh [5],[6],[7]. Parallel strategies, such as data parallelism, tensor parallelism, pipeline parallelism, and sequence parallelism, further optimize performance through task division, alleviating memory and computational resource limitations during model inference [8],[9]. Additionally, many practical applications involve processing long texts, such as multi-turn conversations, and historical data. Models need to capture long-term dependencies to make accurate predictions. The cloud service framework vLLM draws inspiration from the virtual memory management methods of operating systems to optimize key-value cache

* Corresponding Author
Pengyan Zhu is with the Navigation College, Dalian Maritime University, Dalian, China (e-mail: winter_zhu@dlmu.edu.cn).
Tingting Yang is with the Peng Cheng Laboratory, Shenzhen, China, and the Navigation College, Dalian Maritime University, Dalian, China (e-mail: yangtt@pcl.ac.cn).

CE-LSLM：通过云边协作进行高效的大小语言模型推理和通信

朱鹏艳、杨婷婷*

***Abstract*——**6G通信中新兴的智能业务场景对低时延、高可靠性、隐私保护提出了严格的要求。生成大语言模型（LLM）正逐渐成为语义通信和计算集成的关键推动者。然而，由于边缘设备的计算资源有限以及异构终端接入的复杂性不断增加，现有的集中式推理方法无法满足边缘侧推理任务中响应效率和数据隐私的双重需求。为了应对这些挑战，本文提出了一种新颖的协作推理架构，它将基于云的LLM与边缘部署的小语言模型（SLM）相集成，实现语义级中间状态的动态调度和共享，并建立专为6G网络量身定制的统一计算通信范式。具体来说，引入了键值（KV）缓存重用机制，通过云端的上下文指导增强边缘模型的语义理解，同时显着减少边缘侧计算和存储开销。此外，提出了跨节点并行调度机制，实现模型状态加载和解码计算之间的异步协调，从而提高边缘响应能力。此外，我们研究异构模型之间的层对齐和表示压缩策略，以减轻边缘的通信负担。实验结果表明，所提出的架构在推理延迟、系统稳定性和并发处理能力方面表现出优异的适应性和可扩展性。

***Index Terms*——**大语言模型推理； 6G 通信；异构模型协作；隐私保护；资源受限的边缘设备

一、简介

随着第五代（5G）蜂窝系统的大规模商用部署，人们重新渴望探索新的解决方案来解决覆盖范围相对较小、频谱资源有限、设备兼容性差等现实问题，从而改善现有的生活方式。作为5G通信技术的后继者，第六代（6G）将不仅仅是一个传输速率更高、网络容量更大的无线网络，而是一个数字化连接、全自动化、智能化的全球通信网络，能够为不同场景提供多样化的网络服务，包括工业、移动通信、移动通信等。

* Corresponding Author
Pengyan Zhu is with the Navigation College, Dalian Maritime University, Dalian, China (e-mail: winter_zhu@dlmu.edu.cn).
Tingting Yang is with the Peng Cheng Laboratory, Shenzhen, China, and the Navigation College, Dalian Maritime University, Dalian, China (e-mail: yangtt@pcl.ac.cn).

智能、智能可穿戴设备、自动驾驶汽车和智能医疗系统[1]。太赫兹通信是 6G 的基石技术，利用 0.1 THz 至 10 THz 频率范围内的电磁波实现高达 1 Tbps 的数据传输速率。此功能支持专门的 6G 任务场景 [2]、[3]、[4] 的高带宽和超低延迟需求。然而，由于太赫兹波的波长较短，加上其穿透力和传播距离有限，足够的信号覆盖需要部署密集的小规模模型或边缘节点来建立多层网络架构，涵盖近边缘和远边缘分布式场景。从实际角度来看，虽然多层网络架构可以缓解信号覆盖问题，但由于计算能力、内存和带宽资源的限制，边缘节点在直接处理复杂的推理任务时面临着重大挑战。此外，随着设备连接的指数级增长，6G不仅要解决计算和传输资源不足的问题，还要解决强大的用户隐私保护的迫切需求。

以ChatGPT为代表的大语言模型（LLM）的出现将为6G的发展提供重要的驱动力和关键支撑。 GPT系列、BERT等模型拥有数十亿甚至数千亿的参数，具备高精度的自然语言理解和生成能力。他们可以从大量数据中提取更多特征和知识，为解决复杂的现实问题提供更大的潜力。然而，由于这些模型的尺寸巨大，存储和处理其参数需要大量内存。对于资源受限的设备，LLM 的广泛部署具有挑战性。

现有研究通过重构GPU推理集群架构和采用先进的软硬件协同优化技术，在一定程度上提高了LLM的训练和推理效率[5]，[6]，[7]。并行策略，例如数据并行、张量并行、管道并行和序列并行，通过任务划分进一步优化性能，缓解模型推理过程中的内存和计算资源限制[8]，[9]。此外，许多实际应用涉及处理长文本，例如多轮对话和历史数据。模型需要捕获长期依赖性才能做出准确的预测。云服务框架vLLM从操作系统的虚拟内存管理方法中汲取灵感，优化key-value缓存

management and introduces the concept of prefix sharing [10]. By precomputing and storing the attention states of commonly used prompts, studies such as [11] and [12] further extend the application of prefix sharing, enabling efficient reuse of repetitive prompts.

However, the aforementioned studies are primarily focused on cloud computing. On one hand, the extensive data transmission between the cloud and edge devices results in high latency and bandwidth pressure. On the other hand, privacy protection and data security remain challenging in sensitive task scenarios. These limitations of cloud dependency hinder the development of 6G in edge-oriented applications. To address these issues, small models generated through techniques such as fine-tuning and pruning are deployed on edge devices in various regions to share computational tasks and alleviate the burden on the cloud [13], [14]. However, these small models typically require further training, and due to their reduced scale, their output accuracy is often insufficient to fully meet the high-precision inference requirements of complex scenarios.

In modern communication networks, distributed systems are commonly adopted architectures that contribute to reducing the dependence of individual nodes on central servers, thereby enhancing the flexibility and scalability of the network. The concepts of ubiquitous collaboration and interconnectivity have become fundamental principles of 6G. In [15], prompt engineering is introduced into the cloud-edge-terminal collaborative architecture to optimize the quality of service (QoS) for generative AI tasks and to establish efficient collaborative connections. Furthermore, in [16], a hierarchical approach is proposed for LLMs, categorizing them based on computational complexity or resource requirements.

To meet the demands of 6G networks for rapid responsiveness, privacy-preserving communication services, and task diversity, we propose CE-LSLM (Cloud-Edge Large-Small Language Model framework), a novel cloud-edge collaborative inference framework that integrates LLM on the cloud with SLMs at the edge. In this architecture, the cloud is primarily responsible for complex semantic tasks such as long-context modeling, while the edge devices handle lightweight local subtasks with lower computational load. To alleviate the computational and memory burdens on the edge, a cross-device cache reuse mechanism is introduced: edge nodes can reuse deep semantic representations generated by the cloud to enhance inference accuracy, and shallow key-value (KV) caches can be shared among local edge devices to enable collaborative inference and load balancing even in disconnected scenarios. Since most inference processes are executed locally, user data remains on the device, thereby further enhancing privacy protection.

A layer matching strategy based on inter-layer distribution similarity is explored and constructed to align the architectural correspondence between the cloud-based LLM and the edge-deployed SLMs. Meanwhile, an attention-head dimensionality reduction strategy is employed to compress key intermediate caches, thereby reducing transmission and storage overhead. Furthermore, a layer-wise pipelined execution mechanism is designed to enable edge devices to preload intermediate states for subsequent layers while computing the current layer, achieving collaborative parallelism between inference computation and communication loading. This design significantly improves the system's response latency and throughput performance.

In summary, the following key contributions have been made:

- We propose a novel collaborative inference architecture integrating cloud-based LLMs and edge-deployed SLMs, offering a unified computation-communication design tailored to the demands of 6G networks.
- By leveraging both cloud-edge and edge-edge KV cache reuse, combined with a hierarchical and pipelined scheduling strategy, we address the resource constraints of edge devices while meeting complex inference demands in multi-device, multi-task environments.
- We integrate existing layer matching and attention-head compression techniques into a unified cache-sharing framework tailored to heterogeneous LLM-SLM inference. This enables scalable cache reuse across cloud-edge and peer-edge settings, addressing cross-device compatibility and reducing communication overhead in collaborative inference.

We organize the remainder of this article as follows. Section II reviews related work. Section III presents a framework for collaborative cooperation between large and small models in 6G networks. Section IV introduces the system model and communication mechanisms. Section V describes optimization strategies for collaborative inference. Section VI presents experimental evaluations. Section VII concludes the paper. The Appendix provides supplementary materials, such as mathematical proofs.

II. RELATED WORK

Large language models (LLMs), such as GPT-3 and Bloom, typically consist of hundreds of millions to tens of billions of parameters. The resource demands during both training and inference far exceed the memory capacity of a single GPU. To address this challenge, DeepSpeed emphasizes distributed deployment and collaborative mechanisms for multi-GPU inference [7][17]. Its core component, ZeRO (Zero Redundancy Optimizer), partitions gradients, optimizer states, and other model data during distributed training to eliminate redundancy [18]. However, these optimizations primarily target the training phase.

During single-query inference, all attention scores for context tokens must be recomputed for each query, requiring frequent access to intermediate results. This issue is exacerbated in long-sequence tasks, where the complexity of attention computation increases from $O(n)$ to $O(n^2)$, further amplifying computational and memory access redundancy. vLLM addresses this challenge by focusing on the batched processing of multi-user requests, emphasizing the reuse and sharing of cached data to enhance cache management efficiency on a single GPU [10]. Building on this foundation, [**?**] separates system prompts from user prompts, storing intermediate results for system prompts in advance. When computing attention for

管理并引入了前缀共享的概念[10]。通过预先计算和存储常用提示的注意力状态，[11]和[12]等研究进一步扩展了前缀共享的应用，实现了重复提示的高效重用。

然而，上述研究主要集中在云计算方面。一方面，云和边缘设备之间大量的数据传输导致高延迟和带宽压力。另一方面，敏感任务场景下的隐私保护和数据安全仍然具有挑战性。云依赖的这些限制阻碍了 6G 在面向边缘的应用中的发展。为了解决这些问题，通过微调和剪枝等技术生成的小型模型被部署在各个区域的边缘设备上，以共享计算任务并减轻云的负担[13]，[14]。然而，这些小模型通常需要进一步训练，并且由于规模缩小，其输出精度往往不足以完全满足复杂场景的高精度推理要求。

在现代通信网络中，分布式系统是普遍采用的架构，有助于减少各个节点对中央服务器的依赖，从而增强网络的灵活性和可扩展性。无处不在的协作和互联的理念已经成为6G的基本原则。在[15]中，将即时工程引入云-边缘-终端协作架构中，以优化生成式人工智能任务的服务质量（QoS）并建立高效的协作连接。此外，在[16]中，为法学硕士提出了一种分层方法，根据计算复杂性或资源需求对它们进行分类。

为了满足6G网络对快速响应、保护隐私的通信服务和任务多样性的需求，我们提出了CE-LSLM（Cloud-Edge Large-Small Language Model Framework），这是一种新颖的云边协作推理框架，它将云端的LLM与边缘的SLM集成在一起。在这种架构中，云主要负责复杂的语义任务，例如长上下文建模，而边缘设备则处理计算负载较低的轻量级本地子任务。为了减轻边缘的计算和内存负担，引入了跨设备缓存重用机制：边缘节点可以重用云生成的深层语义表示以提高推理精度，并且可以在本地边缘设备之间共享浅层键值（KV）缓存，以便即使在断开连接的情况下也能实现协作推理和负载平衡。由于大多数推理过程都是在本地执行，因此用户数据保留在设备上，从而进一步增强了隐私保护。

探索并构建了一种基于层间分布相似性的层匹配策略，以对齐基于云的LLM和边缘部署的SLM之间的架构对应关系。同时，采用注意力头降维策略来压缩关键中间缓存，从而减少传输和存储开销。此外，还设计了分层流水线执行机制，使边缘设备能够预加载中间状态在计算当前层的同时用于后续层，实现推理计算和通信加载之间的协作并行。这种设计显着改善了系统的响应延迟和吞吐量性能。

总之，做出了以下主要贡献：

- 我们提出了一种新颖的协作推理架构，集成了基于云的 LLM 和边缘部署的 SLM，提供了适合 6G 网络需求的统一计算通信设计。
- 通过利用云边和边边KV缓存重用，结合分层和流水线调度策略，我们解决了边缘设备的资源限制，同时满足多设备、多任务环境中的复杂推理需求。
- 我们将现有的层匹配和注意力头压缩技术集成到一个专为异构 LLM-SLM 推理而定制的统一缓存共享框架中。这使得跨云边缘和对等边缘设置的可扩展缓存重用成为可能，解决了跨设备兼容性问题并减少了协作推理中的通信开销。

我们将本文的其余部分组织如下。第二节回顾相关工作。第三节提出了 6G 网络中大小模型之间协作的框架。第四节介绍系统模型和通信机制。第五节描述了协作推理的优化策略。第六节介绍了实验评估。第七节总结了本文。附录提供了补充材料，例如数学证明。

二.相关工作

大语言模型 (LLM)，例如 GPT-3 和 Bloom，通常由数亿到数百亿个参数组成。训练和推理过程中的资源需求远远超过单个GPU的内存容量。为了应对这一挑战，DeepSpeed强调多GPU推理的分布式部署和协作机制[7][17]。其核心组件ZeRO（零冗余优化器）在分布式训练期间对梯度、优化器状态和其他模型数据进行分区以消除冗余[18]。然而，这些优化主要针对训练阶段。

在单查询推理期间，必须为每个查询重新计算上下文标记的所有注意力分数，需要频繁访问中间结果。这个问题在长序列任务中更加严重，其中注意力计算的复杂性从 $O(n)$ 增加到 $O(n^2)$，进一步放大了计算和内存访问冗余。vLLM 通过专注于多用户请求的批量处理、强调缓存数据的重用和共享来解决这一挑战，以提高单个 GPU 上的缓存管理效率[10]。在此基础上，[?]将系统提示与用户提示分开，提前存储系统提示的中间结果。当计算注意力时

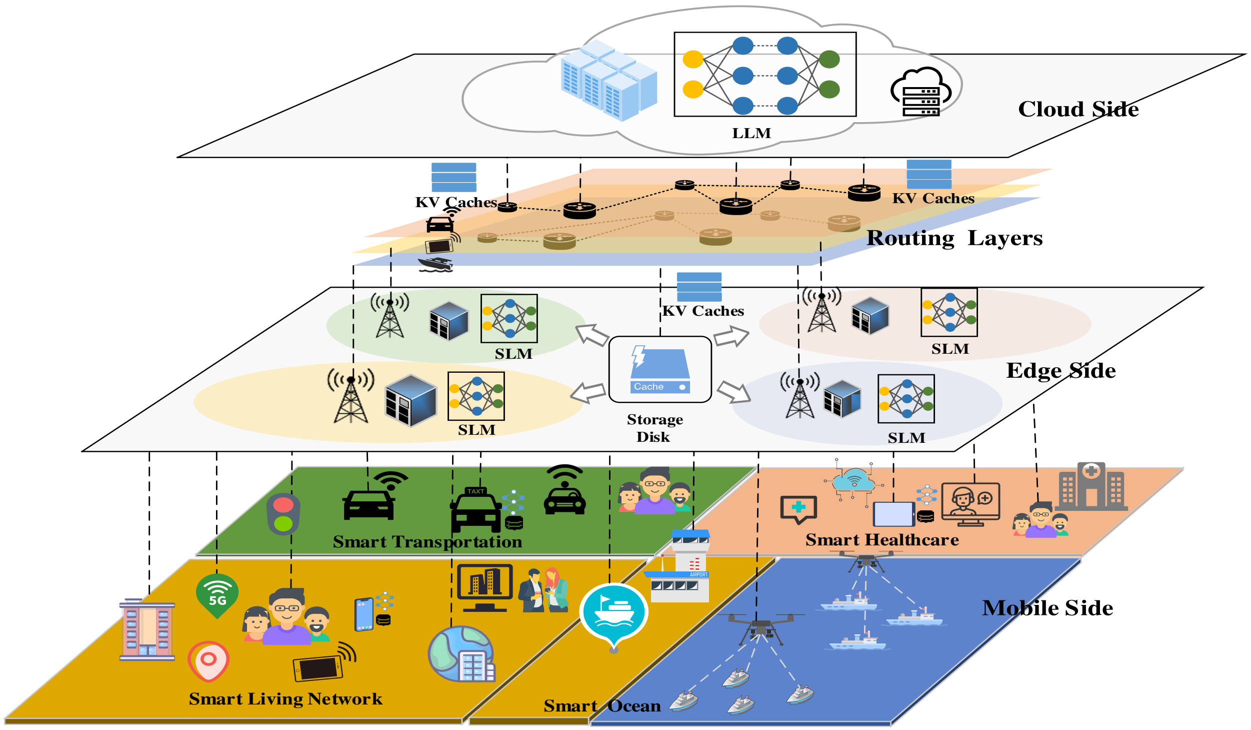


Fig. 1: Collaborative Inference Workflow of Cloud LLM and Edge SLMs Across Heterogeneous Scenarios.

each batch of user prompts, only a single read of the upstream KV cache is required. This approach partially mitigates the redundant memory access issues associated with processing long-context sequences. However, resource-constrained edge devices face significant challenges in handling the computation of system prompts and the storage of intermediate results. As a result, substantial amounts of client data must still be uploaded to the cloud, leading to high latency and increasing the risk of privacy breaches during data transmission.

Lightweight model deployment strategies have made it feasible to implement large language models (LLMs) on resource-constrained edge devices, alleviating the over-reliance on cloud computing. Model compression techniques, including quantization and pruning, are commonly employed to reduce model size and computational complexity. For instance, [19] utilizes gradient information to selectively remove non-critical coupled structures, enabling task-agnostic LLM compression, albeit requiring additional training or re-optimization. In contrast to structured pruning methods, works such as [20], [14], and [21] consider the sensitivity variations across model layers, adapting layer-wise compression strategies accordingly [22]. Cache quantization represents another area of research; [23] and [24] propose adaptive KV cache eviction and allocation mechanisms based on attention head characteristics, reducing KV cache size to fit within predefined memory budgets.

Recently, [16] introduced a cloud-edge collaborative inference architecture, in which certain layers of the LLM are deployed on edge devices while others operate in the cloud. This architecture reduces communication overhead through an early exit mechanism. However, in complex tasks such as intelligent healthcare, the limited computational power of edge devices may prevent the generation of high-confidence inference results. As a result, the frequent transmission of intermediate states between the cloud and edge not only significantly increases communication overhead and latency but also undermines privacy protection. Furthermore, in cases of network interruptions or instability, the cloud-based inference in CE-CoLLM cannot be completed, making it impossible to meet the high accuracy requirements of such tasks. In NetGPT [25], the collaborative inference paradigm is centered around prompt construction, where lightweight LLMs at the edge are responsible for personalized prompt completion. These edge models are further fine-tuned via low-rank adaptation techniques such as LoRA to enhance their semantic understanding.

In contrast, our collaborative architecture is better suited to the diverse and complex task requirements of 6G networks. The cloud-based LLM is primarily responsible for handling long-context inference in domain-specific systems, while edge SLMs focus on processing local user requests with lower resource consumption, ensuring that user privacy data does not need to be uploaded to the cloud. By reusing the cloud's KV cache, SLMs are able to achieve more accurate inference results with reduced computational overhead. Even in the case

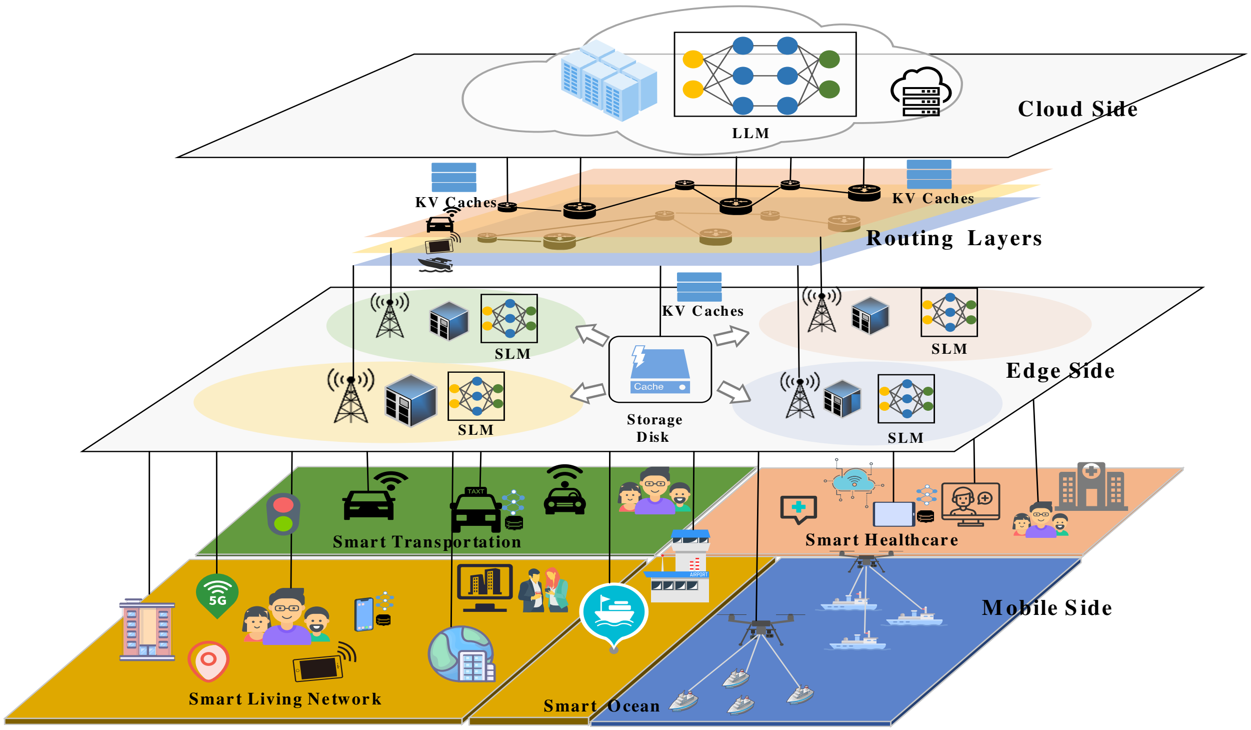


图 1：跨异构场景的云 LLM 和边缘 SLM 的协作推理工作流程。

每一批用户提示，只需要读取一次上游KV缓存。这种方法部分缓解了与处理长上下文序列相关的冗余内存访问问题。然而，资源受限的边缘设备在处理系统提示的计算和中间结果的存储方面面临着巨大的挑战。因此，大量的客户端数据仍然必须上传到云端，从而导致高延迟并增加数据传输过程中隐私泄露的风险。

轻量级模型部署策略使得在资源受限的边缘设备上实现大语言模型（LLM）成为可能，从而减轻了对云计算的过度依赖。模型压缩技术，包括量化和修剪，通常用于减小模型大小和计算复杂性。例如，[19]利用梯度信息有选择地删除非关键耦合结构，从而实现与任务无关的 LLM 压缩，尽管需要额外的训练或重新优化。与结构化剪枝方法相比，[20]、[14]和[21]等工作考虑了模型层之间的敏感性变化，相应地调整了分层压缩策略[22]。缓存量化代表了另一个研究领域；[23]和[24]提出了基于注意力头特征的自适应KV缓存驱逐和分配机制，减少KV缓存大小以适应预定义的内存预算。

最近，[16]引入了一种云边协同推理架构，其中LLM的某些层是部署在边缘设备上，而其他设备则在云中运行。该架构通过提前退出机制减少了通信开销。然而，在智能医疗等复杂任务中，边缘设备有限的计算能力可能会妨碍生成高可信度的推理结果。因此，云端和边缘之间中间状态的频繁传输不仅大大增加了通信开销和延迟，而且还破坏了隐私保护。此外，在网络中断或不稳定的情况下，CE-CoLLM 中的云端推理无法完成，无法满足此类任务的高精度要求。在 NetGPT [25] 中，协作推理范式以提示构建为中心，其中边缘的轻量级 LLM 负责个性化提示完成。这些边缘模型通过 LoRA 等低秩适应技术进一步微调，以增强其语义理解。

相比之下，我们的协作架构更适合6G网络多样化且复杂的任务需求。基于云的LLM主要负责处理特定领域系统中的长上下文推理，而边缘SLM则专注于以较低的资源消耗处理本地用户请求，确保用户隐私数据不需要上传到云端。通过重用云的 KV 缓存，SLM 能够以减少的计算开销获得更准确的推理结果。即使在这种情况下

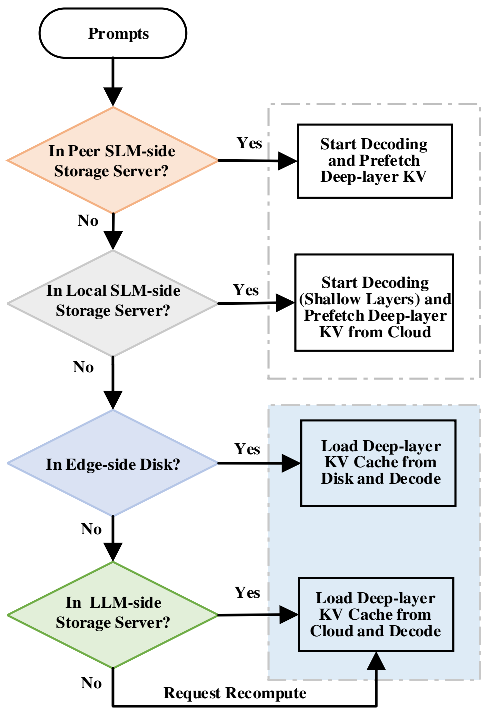


Fig. 2: System architecture illustrating multi-level contextual KV cache scheduling and interaction between SLMs and LLM. The SLM first searches for shallow-layer contextual KV caches from peer or local edge storage to enable early-stage decoding. If a match is found, decoding starts while deeper-layer KV blocks—generated by the cloud LLM—are concurrently loaded from local disk or retrieved from cloud storage. This layer-aware, pipelined mechanism enables fast and resilient inference under dynamic connectivity conditions.

of a cloud-edge network disconnection, the edge devices can still complete inference tasks through collaboration and cache sharing across multiple devices, further enhancing system robustness and inference efficiency.

III. Design and Implementation of a Cloud-Edge Collaborative Inference Architecture

In this section, the fundamental components and overall workflow of the cloud-edge collaborative inference architecture for LLMs and SLMs are first introduced in detail. Subsequently, the dynamic caching strategy is presented, which serves as a key enabler for the efficient allocation, transmission, and utilization of KV caches, acting as a critical bridge for collaborative inference.

A. Framework Overview

In numerous real-world application scenarios, such as intelligent transportation, healthcare, the Internet of Things (IoT), and smart ocean systems, domain-specific task frameworks rely heavily on the correlation analysis of long-term historical data and global inference. Cloud servers are equipped with high-bandwidth GPUs and large-scale memory resources, enabling efficient support for large-scale language models (LLMs) in terms of both parameter loading and long-sequence Key-Value (KV) cache computation and storage. As core intelligent components, LLMs possess powerful capabilities in contextual understanding and generation, allowing them to extract high-level semantic features from input sequences, including long-range dependencies, cross-sentence structures, and implicit task intentions. However, edge devices are significantly constrained in terms of computational power, memory, and storage, making it infeasible to load the full-scale LLM or manage the extensive KV cache required for long-context inference. As a result, Smaller-scale Language Models (SLMs) deployed at the edge are typically responsible for modeling the lower-layer KV caches of system prompts, generating local interactions involving privacy-sensitive user data, and responding quickly to user queries—thereby striking a balance between data security and service latency optimization. Fig. 1 illustrates the basic components and information flow of the architecture.

The cloud-based LLM generates the key-value (KV) cache for the background information (i.e., system prompts) through the attention mechanism, providing contextual support for the inference tasks of SLMs. Based on the model's structural parameters, the generated KV cache is subjected to hierarchical pruning and dimensionality reduction, and it is stored in the cloud service or distributed to the local disk of edge devices to support multi-device sharing and reuse.

The inference process of the edge-side SLM is divided into two stages: on one hand, the shallow-layer KV caches of the system prompt are computed locally, while deep-layer KV caches are progressively loaded from the cloud or local disk. On the other hand, once the KV cache for a given layer of the system prompt is prepared, the corresponding Q, K, and V computations for the user prompt are immediately executed. Through reserved encoding positions, the KV caches from both local and remote sources are concatenated into a complete contextual KV sequence, which is then used to generate the attention output for that layer. As the deep-layer cache loading is completed, the computation concludes. The final inference result is then returned to the user. The cache loading process is illustrated in Fig. 2.

The design of this architecture is based on several key considerations. First, loading deep-layer KV caches helps to enhance the semantic information of edge outputs and alleviate the computational burden on the SLM, while shallow-layer KV caches, being more sensitive to generative behaviors, need to be generated locally by the edge SLM. Second, local computation is typically faster than remote KV loading, which helps to accelerate decoding initiation. Meanwhile, edge devices do not require the preloading of all cloud-based KV caches, significantly alleviating storage pressure [26]. Therefore, a layer-wise pipelined decoding mechanism is adopted, enabling parallel execution of computation and cache loading, as well as coordinated processing of system prompts and user requests, which effectively accelerates user-prompt

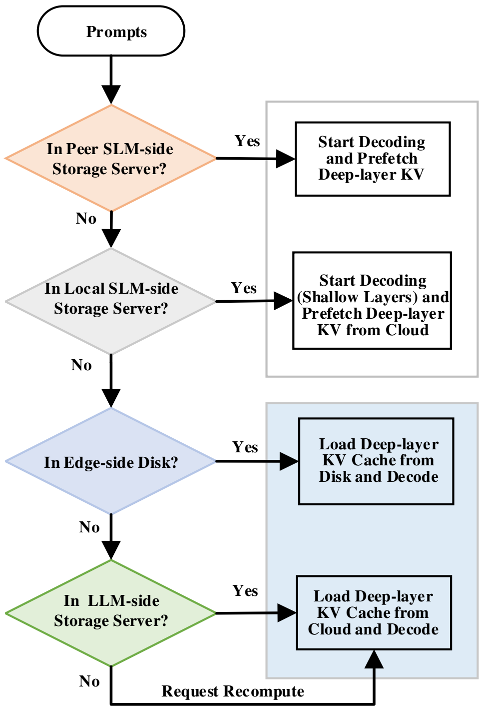


图 2：系统架构说明了多级上下文 KV 缓存调度以及 SLM 和 LLM 之间的交互。 SLM 首先从对等或本地边缘存储搜索浅层上下文 KV 缓存，以实现早期解码。如果找到匹配，则开始解码，同时从本地磁盘加载或从云存储检索由云 LLM 生成的更深层 KV 块。这种分层感知的流水线机制可在动态连接条件下实现快速且有弹性的推理。

在云边网络断线的情况下，边缘设备仍然可以通过多设备协作和缓存共享来完成推理任务，进一步增强系统的鲁棒性和推理效率。

三．云边缘协同推理架构的设计与实现

在本节中，首先详细介绍LLM和SLM的云边协同推理架构的基本组件和总体工作流程。随后，提出了动态缓存策略，该策略是KV缓存高效分配、传输和利用的关键推动者，也是协作推理的关键桥梁。

A. Framework Overview

在智能交通、医疗保健、物联网（IoT）和智能海洋系统等众多实际应用场景中，特定领域的任务框架严重依赖长期历史数据的相关性分析和全局推断。云服务器配备高带宽GPU和大规模内存资源，在参数加载和长序列Key-Value（KV）缓存计算和存储方面能够高效支持大规模语言模型（LLM）。作为核心智能组件，LLM 拥有强大的上下文理解和生成能力，使其能够从输入序列中提取高级语义特征，包括长程依赖、跨句子结构和隐式任务意图。然而，边缘设备在计算能力、内存和存储方面受到严重限制，因此无法加载全面的 LLM 或管理长上下文推理所需的大量 KV 缓存。因此，部署在边缘的小型语言模型（SLM）通常负责对系统提示的下层 KV 缓存进行建模，生成涉及隐私敏感的用户数据的本地交互，并快速响应用户查询，从而在数据安全和服务延迟优化之间取得平衡。图 1 说明了该架构的基本组件和信息流。

基于云的LLM通过注意力机制为背景信息（即系统提示）生成键值（KV）缓存，为SLM的推理任务提供上下文支持。根据模型的结构参数，对生成的KV缓存进行分层剪枝和降维，存储在云服务中或分布到边缘设备的本地磁盘上，以支持多设备共享和复用。

边缘侧SLM的推理过程分为两个阶段：一方面，系统提示的浅层KV缓存在本地计算，而深层KV缓存则从云端或本地磁盘逐步加载。另一方面，一旦准备好系统提示给定层的KV缓存，就立即执行用户提示的相应Q、K和V计算。通过保留的编码位置，来自本地和远程源的 KV 缓存被连接成一个完整的上下文 KV 序列，然后用于生成该层的注意力输出。当深层缓存加载完成后，计算结束。最终的推理结果返回给用户。缓存加载过程如图2所示。

该架构的设计基于几个关键考虑因素。首先，加载深层KV缓存有助于增强边缘输出的语义信息并减轻SLM的计算负担，而浅层KV缓存对生成行为更加敏感，需要由边缘SLM在本地生成。其次，本地计算通常比远程 KV 加载更快，这有助于加速解码启动。同时，边缘设备不需要预加载所有基于云的KV缓存，显着减轻了存储压力[26]。因此，采用分层流水线解码机制，实现计算和缓存加载的并行执行，以及系统提示和用户请求的协调处理，有效加速用户提示

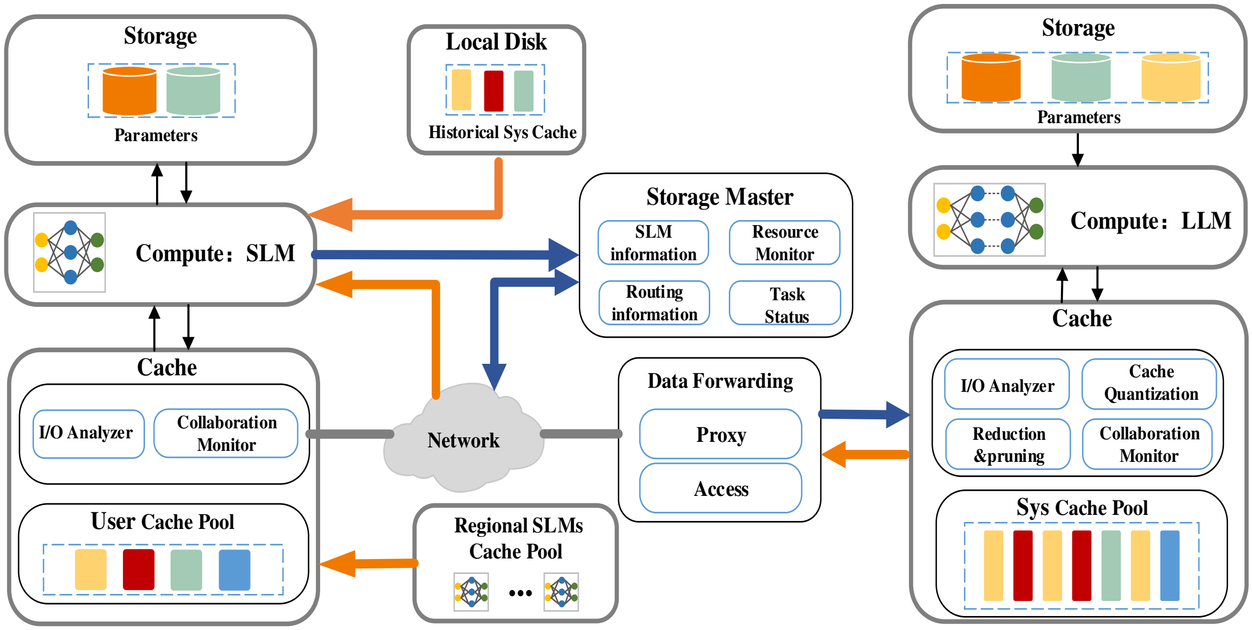


Fig. 3: Schematic of Cache and Storage System Interaction Between LLM and SLM. From left to right, the diagram illustrates the edge device storage and caching module, the data forwarding layer, and the cloud storage and caching module. During the execution of complex inference tasks, the edge SLM requests and downloads the optimized contextual KV cache from the cloud-based LLM.

decoding and enhances overall inference performance.

B. Dynamic Cache Management System

In traditional communication systems, the core value lies in reliably transmitting raw or processed data from the sender to the receiver while ensuring data integrity and consistency. However, KV caches, as intermediate states in model inference, encapsulate substantial semantic information. They are not merely data but also directly usable inference units. This transforms communication networks from simple data forwarding systems into collaborative communication frameworks that support intelligent inference and generation. Within this framework, the cache management system becomes a critical component, responsible for the dynamic allocation and optimization of KV caches to ensure efficient collaboration between the cloud and edge models. Due to the differing functional roles of the cloud and edge in collaborative inference, the design of their cache management systems also varies, as illustrated in Fig. 3.

The Storage Server is primarily responsible for the long-term storage of core data such as model weight parameters, whereas the Cache Server is utilized for temporarily storing Key-Value (KV) caches of prompts. Its submodules include the Collaboration Monitor, which continuously monitors edge requests and system coordination status to ensure efficient cache allocation, and the I/O Analyzer, which analyzes the input/output access patterns of cached data. In addition, a cache optimization module is employed to dynamically adjust the quantization precision and prune redundant layers or dimensions in order to accommodate the inference requirements of both LLMs and SLMs. The storage and caching modules on the edge are designed similarly to those on the cloud. However, in the edge-side Local Cache, a historical cache module is deployed to store system prompt KV caches periodically downloaded from the cloud. This mechanism allows the cached content to serve as backup context during network disconnection, thereby maintaining inference continuity.

The communication system consists of two modules: Storage Master and Data Forwarding. The Storage Master provides information such as the structure of the edge SLM and network bandwidth, which is then synchronized with the Proxy module. The Proxy module is responsible for making transmission path decisions, selecting either direct (point-to-point) transmission or routing data to the cloud, based on the network status. In case of network anomalies, the Proxy retrieves the context cache from the edge device's disk, utilizing the local cache to complete the inference. The Access module is responsible for the actual data transmission tasks [27].

IV. SYSTEM MODEL AND OPTIMIZE CLOUD-EDGE INFERENCE

This section focuses on the key computational mechanisms of cloud-edge collaboration and an analysis of communication performance. First, the layer-by-layer attention computation and KV cache reuse mechanisms, which form the foundational basis of cloud-edge collaborative inference, are described in

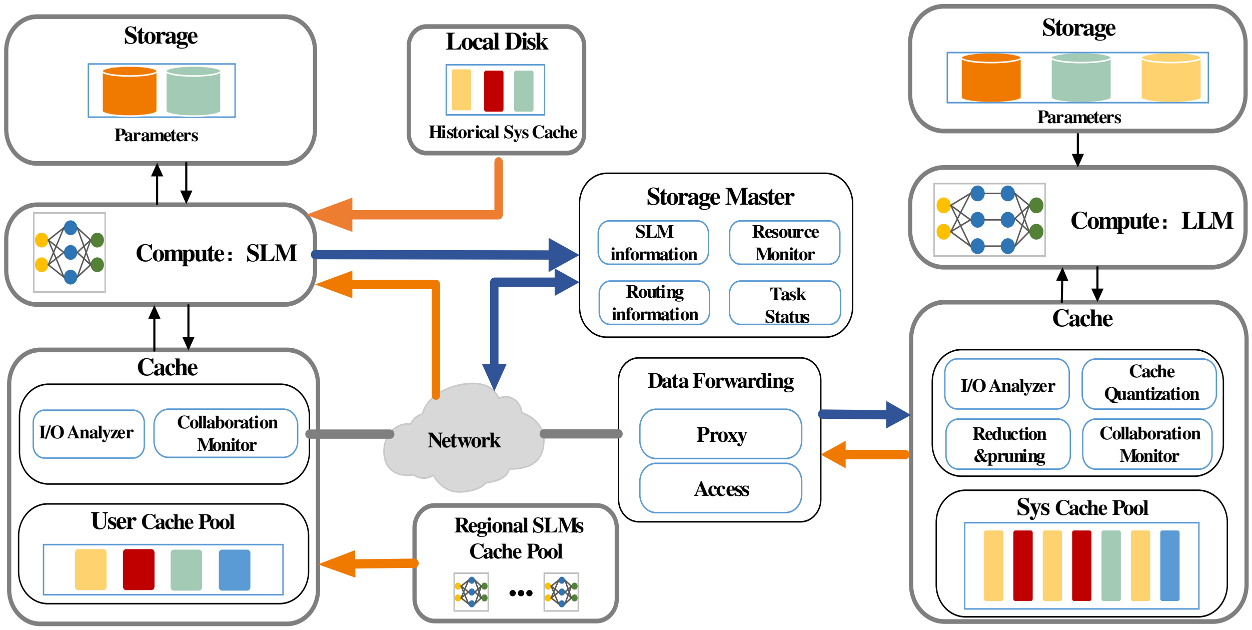


图3：LLM和SLM之间的缓存和存储系统交互示意图。该图从左到右依次为边缘设备存储和缓存模块、数据转发层、云存储和缓存模块。在执行复杂推理任务期间，边缘SLM从基于云的LLM请求并下载优化的上下文KV缓存。

解码并增强整体推理性能。

B. Dynamic Cache Management System

在传统通信系统中，核心价值在于将原始或处理后的数据从发送方可靠地传输到接收方，同时确保数据的完整性和一致性。然而，KV 缓存作为模型推理中的中间状态，封装了大量的语义信息。它们不仅仅是数据，也是直接可用的推理单元。这将通信网络从简单的数据转发系统转变为支持智能推理和生成的协作通信框架。在此框架内，缓存管理系统成为关键组件，负责KV缓存的动态分配和优化，以确保云端和边缘模型之间的高效协作。由于云和边缘在协同推理中的功能角色不同，其缓存管理系统的设计也有所不同，如图3所示。

存储服务器主要负责模型权重参数等核心数据的长期存储，而缓存服务器则用于临时存储提示的键值（KV）缓存。其子模块包括协作监视器（持续监视边缘请求和系统协调状态以确保高效的缓存分配）和 I/O 分析器（分析缓存数据的输入/输出访问模式）。此外，还采用了缓存优化模块来动态调整量化精度并修剪冗余层或维度，以适应 LLM 和 SLM 的推理要求。边缘的存储和缓存模块的设计与云端类似。然而，在边缘侧本地缓存中，部署了历史缓存模块来存储定期从云端下载的系统提示KV缓存。这种机制允许缓存的内容在网络断开时作为备份上下文，从而保持推理的连续性。

通信系统由两个模块组成：存储主模块和数据转发模块。 Storage Master提供边缘SLM的结构和网络带宽等信息，然后与Proxy模块同步。 Proxy模块负责根据网络状态做出传输路径决策，选择直接（点对点）传输或将数据路由到云端。当网络异常时，Proxy从边缘设备的磁盘中检索上下文缓存，利用本地缓存完成推理。 Access模块负责实际的数据传输任务[27]。

四. 系统模型和云边缘推理优化

本节重点介绍云边协作的关键计算机制以及通信性能分析。首先，逐层注意力计算和KV缓存重用机制是云边协同推理的基础，描述如下：

detail. Subsequently, a communication performance model is proposed to quantify the time delay resulting from data transmission during collaboration. This model provides a theoretical basis for developing efficient transmission strategies in 6G networks.

A. System Model

A causal encoder adopts a decoder-only Transformer architecture and is widely used in autoregressive generation tasks, such as text generation and language modeling. In this architecture, the decoder predicts and generates one token at each time step, relying solely on previously generated tokens as contextual information, with no access to future positions. Leveraging this property, the representations generated from the system prompt can be reused as contextual inputs for user prompt inference. By appropriately partitioning the inference responsibilities of system and user prompts, the collaborative inference framework enables upstream–downstream decoupling and computational load balancing.

We consider a single attention head where the system prompt has a length of s, the calculations of the query, key, and value for the l-th layer, for all $l \in \{1, 2, \dots, N\}$, are expressed as follows :

$$\begin{aligned} q_{ctx,i}^{(l)} &= x_{ctx,i}^{(l)} \cdot W_Q^{(l)} \\ k_{ctx,i}^{(l)} &= x_{ctx,i}^{(l)} \cdot W_K^{(l)} \\ v_{ctx,i}^{(l)} &= x_{ctx,i}^{(l)} \cdot W_V^{(l)} \end{aligned} \tag{1}$$

where $i \in \{1, 2, \dots, s\}$ indexes the position of tokens within the system prompt sequence, and $x_{ctx}^{(0)} = \gamma \cdot r_{ctx} + b$ denotes the transformed input features r_{ctx} after applying linear transformations (e.g., scaling and bias). W_Q, W_K and W_V represent the query, key and value weight matrices of each attention head in the cloud model.

Then, the attention output can be defined as:

$$\begin{aligned} o_s^{(l)} &= Attention\left(q_s^{(l)}, \left\{k_i^{(l)}\right\}_{i=1}^{s}, \left\{v_i^{(l)}\right\}_{i=1}^{s}\right) \\ &= \sum_{i=1}^{s} \frac{exp\left(q_s^{(l)}\left(k_i^{(l)}\right)^T\right)}{\sigma_{1\to s}^{(l)}} v_i^{(l)} \end{aligned} \tag{2}$$

where $\sigma_{1\to s}^{(l)} = \sum_{i=1}^{s} exp\left(q_s^{(l)}\left(k_i^{(l)}\right)^T\right)$ is used to normalize the attention weights. the output $o_s^{(l)}$ will serve as the input to the next layer, continuing the attention calculation until the entire inference process is completed.

The cloud model stores the intermediate results $k_{ctx}^{(l)}$ and $v_{ctx}^{(l)}$ for each layer, which are made available for SLMs to download and utilize. In addition, within a local region, multiple SLMs can collaboratively compute or share these contextual representations, thereby reducing redundant computation and improving memory efficiency. Meanwhile, the output serves as the input to the next layer $x_{ctx,s}^{(l+1)} = o_s^{(l)}$, enabling the continuation of attention computation until the inference process is completed.

Based on system prompt processing, the edge-side SLM is responsible for the localized computation and caching of the user prompt and the partially generated response tokens. Let the user prompt have a length of u, and assume that $t-1$ response tokens have already been generated. Then, the tokens from position $s+1$ to $L = s+u+t-1$ participate in the current attention computation. The corresponding query, key, and value representations are computed as follows:

$$\begin{aligned} q_{user,j}^{(l)} &= x_{user,j}^{(l)} \cdot W_{Q,SLM}^{(l)} \\ k_{user,j}^{(l)} &= x_{user,j}^{(l)} \cdot W_{K,SLM}^{(l)} \\ v_{user,j}^{(l)} &= x_{user,j}^{(l)} \cdot W_{V,SLM}^{(l)} \end{aligned} \tag{3}$$

The attention output can be defined as:

$$\begin{aligned} o_L^{(l)} &= Attention\left(q_L^{(l)}, \left\{k_j^{(l)}\right\}_{j=s+1}^{L}, \left\{v_j^{(l)}\right\}_{j=s+1}^{L}\right) \\ &= \sum_{j=s+1}^{L} \frac{exp\left(q_L^{(l)}\left(k_j^{(l)}\right)^T\right)}{\sigma_{s+1\to L}^{(l)}} v_j^{(l)} \end{aligned} \tag{4}$$

where $\sigma_{s+1\to L}^{(l)} = \sum_{j=s+1}^{L} exp\left(q_L^{(l)}\left(k_j^{(l)}\right)^T\right)$ is used to normalize the attention weights.

In summary, for a batch of inference requests originating from user devices, the edge server coordinates the collaborative inference process between the cloud-based LLM and the local SLMs. At generation step $t = L+1$, the attention output o_t of SLM is defined as follows:

$$\begin{aligned} o_{t,SLM} = &\alpha_{ctx}^t Atttention\left(q_t, \{k_i\}_{i=1}^{s}, \{v_i\}_{i=1}^{s}\right) + \\ &\alpha_{user}^t Atttention\left(q_t, \{k_j\}_{j=s+1}^{L}, \{v_j\}_{j=s+1}^{L}\right) \\ s.t.\ &\alpha_{ctx}^t = \frac{\sigma_{1\to s}^{(l)}}{\sigma_{1\to L}^{(l)}},\ \alpha_{user}^t = \frac{\sigma_{s+1\to L}^{(l)}}{\sigma_{1\to L}^{(l)}} \\ &\alpha_{ctx}^t + \alpha_{usr}^t = 1 \end{aligned} \tag{5}$$

This hierarchical collaborative inference architecture offers significant advantages. On one hand, resource-constrained

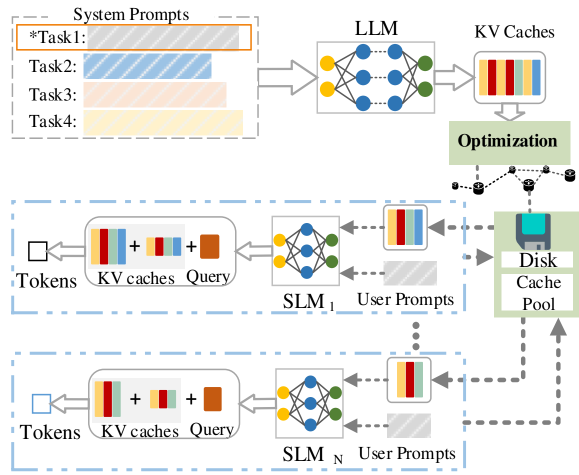


Fig. 4: Illustration of Network-Resilient LLM Inference via Cache Reuse and Token Generation in Edge SLMs.

细节。随后，提出了一种通信性能模型来量化协作期间数据传输造成的时间延迟。该模型为制定6G网络高效传输策略提供了理论基础。

A. System Model

因果编码器采用仅解码器的 Transformer 架构，广泛应用于自回归生成任务，例如文本生成和语言建模。在此架构中，解码器在每个时间步预测并生成一个令牌，仅依赖于先前生成的令牌作为上下文信息，而无法访问未来的位置。利用此属性，从系统提示生成的表示可以重新用作用户提示推理的上下文输入。通过适当划分系统和用户提示的推理职责，协作推理框架可以实现上下游解耦和计算负载平衡。

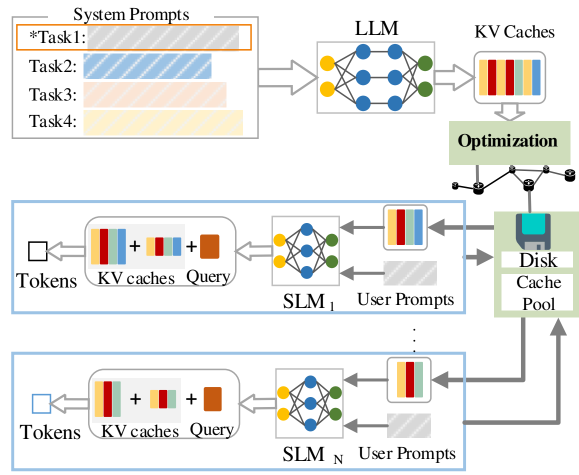


图 4：通过边缘 SLM 中的缓存重用和令牌生成进行网络弹性 LLM 推理的图示。

我们考虑单个注意力头，其中系统提示的长度为 s，对于所有 $l \in \{1, 2, \ldots, N\}$，第 l 层的查询、键和值的计算表示如下：

$$\begin{aligned} q_{ctx,i}^{(l)} &= x_{ctx,i}^{(l)} \cdot W_Q^{(l)} \\ k_{ctx,i}^{(l)} &= x_{ctx,i}^{(l)} \cdot W_K^{(l)} \\ v_{ctx,i}^{(l)} &= x_{ctx,i}^{(l)} \cdot W_V^{(l)} \end{aligned} \tag{1}$$

其中 $i \in \{1, 2, \ldots, s\}$ 索引系统提示序列中标记的位置，而 $x_{ctx}^{(0)} = \gamma \cdot r_{ctx} + b$ 表示应用线性变换（例如缩放和偏差）后变换后的输入特征 r_{ctx}。W_Q、W_K和W_V表示云模型中每个注意力头的查询、键和值权重矩阵。

那么，注意力输出可以定义为：

$$\begin{aligned} o_s^{(l)} &= Attention\left(q_s^{(l)}, \left\{k_i^{(l)}\right\}_{i=1}^{s}, \left\{v_i^{(l)}\right\}_{i=1}^{s}\right) \\ &= \sum_{i=1}^{s} \frac{exp\left(q_s^{(l)}\left(k_i^{(l)}\right)^T\right)}{\sigma_{1\to s}^{(l)}} v_i^{(l)} \end{aligned} \tag{2}$$

其中 $\sigma_{1\to s}^{(l)} = \sum_{i=1}^{s} exp\left(q_s^{(l)}\left(k_i^{(l)}\right)^T\right)$ 用于标准化注意力权重。输出$o_s^{(l)}$将作为下一层的输入，继续注意力计算，直到整个推理过程完成。

云模型存储每一层的中间结果$k_{ctx}^{(l)}$和$v_{ctx}^{(l)}$，可供SLM下载和使用。此外，在局部区域内，多个SLM可以协作计算或共享这些上下文表示，从而减少冗余计算并提高内存效率。同时，输出作为下一层$x_{ctx,s}^{(l+1)} = o_s^{(l)}$的输入，使得注意力计算能够继续，直到推理过程完成。

基于系统提示处理，边缘侧SLM负责局部计算和缓存用户提示和部分生成的响应令牌。让用户提示的长度为u，并假设已生成$t-1$ 个响应令牌。然后，位置$s+1$ 到 $L = s + u + t - 1$ 的标记参与当前的注意力计算。相应的查询、键和值表示计算如下：

$$\begin{aligned} q_{user,j}^{(l)} &= x_{user,j}^{(l)} \cdot W_{Q,SLM}^{(l)} \\ k_{user,j}^{(l)} &= x_{user,j}^{(l)} \cdot W_{K,SLM}^{(l)} \\ v_{user,j}^{(l)} &= x_{user,j}^{(l)} \cdot W_{V,SLM}^{(l)} \end{aligned} \tag{3}$$

注意力输出可以定义为：

$$\begin{aligned} o_L^{(l)} &= Attention\left(q_L^{(l)}, \left\{k_j^{(l)}\right\}_{j=s+1}^{L}, \left\{v_j^{(l)}\right\}_{j=s+1}^{L}\right) \\ &= \sum_{j=s+1}^{L} \frac{exp\left(q_L^{(l)}\left(k_j^{(l)}\right)^T\right)}{\sigma_{s+1\to L}^{(l)}} v_j^{(l)} \end{aligned} \tag{4}$$

其中 $\sigma_{s+1\to L}^{(l)} = \sum_{j=s+1}^{L} exp\left(q_L^{(l)}\left(k_j^{(l)}\right)^T\right)$ 用于标准化注意力权重。

总之，对于来自用户设备的一批推理请求，边缘服务器协调基于云的LLM和本地SLM之间的协作推理过程。在生成步骤$t = L + 1$，SLM的注意力输出o_t定义如下：

$$\begin{aligned} o_{t,SLM} = &\alpha_{ctx}^{t} Atttention\left(q_t, \{k_i\}_{i=1}^{s}, \{v_i\}_{i=1}^{s}\right) + \\ &\alpha_{user}^{t} Attention\left(q_t, \{k_j\}_{j=s+1}^{L}, \{v_j\}_{j=s+1}^{L}\right) \\ s.t.\ &\alpha_{ctx}^{t} = \frac{\sigma_{1\to s}^{(l)}}{\sigma_{1\to L}^{(l)}},\ \alpha_{user}^{t} = \frac{\sigma_{s+1\to L}^{(l)}}{\sigma_{1\to L}^{(l)}} \\ &\alpha_{ctx}^{t} + \alpha_{usr}^{t} = 1 \end{aligned} \tag{5}$$

这种分层协作推理架构具有显着的优势。一方面，资源有限

edge models can avoid repeatedly computing long system prompts, thereby improving the throughput for handling user requests. On the other hand, the KV cache of the system prompt can be reused across multiple SLMs instances, significantly enhancing overall resource efficiency. Fig. 4 illustrates the workflow of a specific task (e.g., Task 1), in which multiple edge SLMs share the same system prompt KV cache and integrate it with user prompt caches to generate personalized outputs.

B. Communication Performance

In cloud–edge collaborative inference, communication overhead caused by task partitioning is as critical as computational load, and this trade-off becomes more prominent under 6G network conditions. The inference latency of LLMs is commonly estimated based on the total number of $FLOPs$ and the volume of I/O bytes transferred. Hence, we have:

$$T_{com_C} = \sum_{l=0}^{N-1} t_{comp_C}^{(l)} = \sum_{l=0}^{N-1} t_{FLOPs}^{(l)} + t_{I/O}^{(l)} + t_{decode}^{(l)} \quad (6)$$

where N denotes the total number of layers in LLM, $t_{FLOPs}^{(l)} = \frac{FLOPs}{TFLOPs/s}$ and $t_{I/O}^{(l)} = \frac{I/O\ bytes}{B_m}$ represent the floating-point computation time and the data input/output time for the i-th layer on the edge device, respectively. B_m is Memory Bandwidth.

Similarly, the inference time of the SLM is expressed as:

$$T_{com_E} = \sum_{l=0}^{M-1} t_{comp_E}^{(l)} = \sum_{l=0}^{M-1} t_{FLOPs}^{(l)} + t_{I/O}^{(l)} + t_{decode}^{(l)} \quad (7)$$

where M denotes the total number of layers in SLM.

The transmission latency between cloud–edge and edge–edge devices is affected by the available bandwidth and can be expressed as:

$$T_{comm} = \sum_{l=0}^{N-1} t_{comm}^{(l)} = \sum_{l=0}^{N-1} \frac{D^{(l)}}{B_t^{(l)}} \quad (8)$$

where D denotes the size of kv cache, and B_t represents the network bandwidth at the current time step.

The total inference time can thus be calculated by

$$T_{total} = T_{com_C} + T_{comm} + T_{com_E} \quad (9)$$

The ultra-low latency requirements of 6G networks emphasize the need for real-time responsiveness. Through the cloud-edge collaborative inference architecture, we can achieve fast responses to a large number of user requests while ensuring the privacy protection of local data. Therefore, the problem of minimizing inference latency can be formulated as follows:

$$\begin{aligned} &min\ T_{total} \\ &s.t.\ \max\left(\{Req_l \mid 1 \le l \le N\}\right) \le Mem_E \\ &\qquad \max\left\{O_l \mid 1 \le l \le N\right\} \le B_l \times T_{max} \end{aligned} \quad (10)$$

where Req_l represents the memory required for the l-th layer's KV caches in cloud, and Mem_E represents the available memory in the edge SLM. Additionally, o_l represents the amount of data to be transmitted, B_l denotes the transmission bandwidth, and T_{max} is the maximum tolerable transmission delay.

V. Optimize Communication Transmission latency

This section presents a set of techniques aimed at alleviating the computational and communication bottlenecks in collaborative inference. By introducing layer-wise matching and attention head dimensionality reduction, cache reuse between heterogeneous models becomes feasible, allowing the cloud model to enhance the contextual understanding capability of edge models without increasing their computational burden. Furthermore, by optimizing the scheduling of computation and cache loading, the framework enables joint acceleration of computation and transmission, thereby significantly reducing inference latency and improving overall system throughput.

A. Layer-wise Similarity Matching for Cloud–Edge Models

As system prompts vary across tasks, they influence the activation patterns of LLM, leading to differences in the statistical distributions of KV caches across layers. Prior studies have shown that KV caches from structurally similar layers can be interchangeable without significantly affecting the final hidden state outputs [21]. Motivated by this, we propose an exploratory layer-wise matching strategy based on representational similarity, aiming to facilitate KV cache reuse in cloud–edge collaborative inference.Specifically, we select n out of N layers of LLM and transmitted to SLM. The hidden features of the remaining layers in the LLM to provide long-context KV caches for the SLMs, while the remaining $M-n$ layers are computed locally by the current SLM or by neighboring SLMs of the same type, thereby enabling adaptive cache sharing under dynamic and heterogeneous tasks.

Specifically, when comparing the spatial structures of two layers, directly comparing their raw activation values is often affected by factors such as permutation, scale, and transformation. To address this, representational similarity matrices (RSMs) [28] are employed to capture the structural similarity across representations by computing the similarity between each instance ii and all other instances. The mathematical formulation is given as [28]:

$$S_{ij}^{(l)} := s(O_i^{(l)}, O_j^{(l)}) \quad (11)$$

Here, O_i and O_j denote the i-th and j-th rows of the output matrix $O \in \mathbb{R}^{N\times D}$ from the $l-th$ layer, where N s the number of input samples and D is the hidden size of the current layer. $S \in \mathbb{R}^{N\times N}$ is a symmetric matrix, and $s(\cdot,\cdot)$ represents a generic similarity function. In the following, we jointly evaluate the similarity between heterogeneous models from both spatial and behavioral perspectives.

Compared with other similarity measures, Centered Kernel Alignment (CKA) possesses several desirable invariance properties, with their theoretical proof provided in Appendix A. Therefore, CKA is more suitable for analyzing the intrinsic structural similarity between layer-wise representations in neural networks, rather than superficial differences such as numerical scaling or neuron ordering.

HSIC is used to quantify the statistical dependence between two kernel matrices, where a larger value indicates greater

边缘模型可以避免重复计算较长的系统提示，从而提高处理用户请求的吞吐量。另一方面，系统提示符的KV缓存可以在多个SLM实例之间重用，从而显着提高整体资源效率。图4说明了特定任务（例如任务1）的工作流程，其中多个边缘SLM共享相同的系统提示KV缓存并将其与用户提示缓存集成以生成个性化输出。

B. Communication Performance

在云边协同推理中，任务划分引起的通信开销与计算负载一样重要，并且这种权衡在6G网络条件下变得更加突出。 LLM 的推理延迟通常根据 $FLOPs$ 总数和传输的 I/O 字节量来估计。因此，我们有：

$$T_{com_C} = \sum_{l=0}^{N-1} t_{comp_C}^{(l)} = \sum_{l=0}^{N-1} t_{FLOPs}^{(l)} + t_{I/O}^{(l)} + t_{decode}^{(l)} \tag{6}$$

其中N表示LLM中的总层数，$t_{FLOPs}^{(l)} = \frac{FLOPs}{TFLOPs/s}$和$t_{I/O}^{(l)} = \frac{I/O_bytes}{B_m}$分别表示边缘设备上第$i$层的浮点计算时间和数据输入/输出时间。 B_m 是内存带宽。

类似地，SLM的推理时间表示为：

$$T_{com_E} = \sum_{l=0}^{M-1} t_{comp_E}^{(l)} = \sum_{l=0}^{M-1} t_{FLOPs}^{(l)} + t_{I/O}^{(l)} + t_{decode}^{(l)} \tag{7}$$

其中 M 表示 SLM 中的总层数。

云-边和边-边设备之间的传输延迟受可用带宽的影响，可以表示为：

$$T_{comm} = \sum_{l=0}^{N-1} t_{comm}^{(l)} = \sum_{l=0}^{N-1} \frac{D^{(l)}}{B_t^{(l)}} \tag{8}$$

其中D表示kv缓存的大小，B_t表示当前时间步的网络带宽。

因此，总推理时间可以通过以下方式计算：

$$T_{total} = T_{com_C} + T_{comm} + T_{com_E} \tag{9}$$

6G网络的超低时延要求强调了对实时响应能力的需求。通过云边协同推理架构，可以实现对大量用户请求的快速响应，同时保证本地数据的隐私保护。因此，最小化推理延迟的问题可以表述如下：

$$\begin{aligned} &min\ T_{total} \\ &s.t.\ \max\left(\{Req_l \mid 1 \le l \le N\}\right) \le Mem_E \\ &\qquad \max\left\{O_l \mid 1 \le l \le N\right\} \le B_l \times T_{max} \end{aligned} \tag{10}$$

其中，Req_l 表示云中第 l 层 KV 缓存所需的内存，Mem_E 表示边缘 SLM 中的可用内存。另外，o_l表示要传输的数据量，B_l表示传输带宽，T_{max}是最大可容忍的传输延迟。

五、优化通信传输延迟

本节介绍了一系列旨在缓解协作推理中的计算和通信瓶颈的技术。通过引入逐层匹配和注意力头降维，异构模型之间的缓存重用变得可行，使得云模型能够在不增加边缘模型计算负担的情况下增强边缘模型的上下文理解能力。此外，通过优化计算和缓存加载的调度，该框架可以实现计算和传输的联合加速，从而显着减少推理延迟并提高整体系统吞吐量。

A. Layer-wise Similarity Matching for Cloud–Edge Models

由于不同任务的系统提示不同，它们会影响LLM的激活模式，导致跨层KV缓存的统计分布存在差异。先前的研究表明，结构相似层的 KV 缓存可以互换，而不会显着影响最终的隐藏状态输出 [21]。受此启发，我们提出了一种基于表征相似性的探索性逐层匹配策略，旨在促进云边协同推理中的KV缓存重用。具体来说，我们从LLM的N层中选择n并传输到SLM。 LLM中剩余层的隐藏特征为SLM提供长上下文KV缓存，而剩余的$M-n$层由当前SLM或相同类型的相邻SLM本地计算，从而实现动态和异构任务下的自适应缓存共享。

具体来说，在比较两层的空间结构时，直接比较它们的原始激活值通常会受到排列、尺度和变换等因素的影响。为了解决这个问题，采用表征相似性矩阵（RSM）[28]通过计算每个实例 ii 与所有其他实例之间的相似性来捕获表征之间的结构相似性。数学公式如[28]所示：

$$S_{ij}^{(l)} := s(O_i^{(l)}, O_j^{(l)}) \tag{11}$$

这里，O_i 和 O_j 表示来自 $l-th$ 层的输出矩阵 $O \in \mathbb{R}^{N\times D}$ 的第 i 行和第 j 行，其中 N 是输入样本的数量，D 是当前层的隐藏大小。 $S \in \mathbb{R}^{N\times N}$是对称矩阵，$s(\cdot,\cdot)$表示通用相似度函数。下面，我们从空间和行为角度共同评估异构模型之间的相似性。

与其他相似性度量相比，中心核对齐（CKA）具有几个理想的不变性，其理论证明在附录A中提供。因此，CKA 更适合分析神经网络中逐层表示之间的内在结构相似性，而不是诸如数值缩放或神经元排序等表面差异。

HSIC用于量化两个核矩阵之间的统计依赖性，其中值越大表示越大

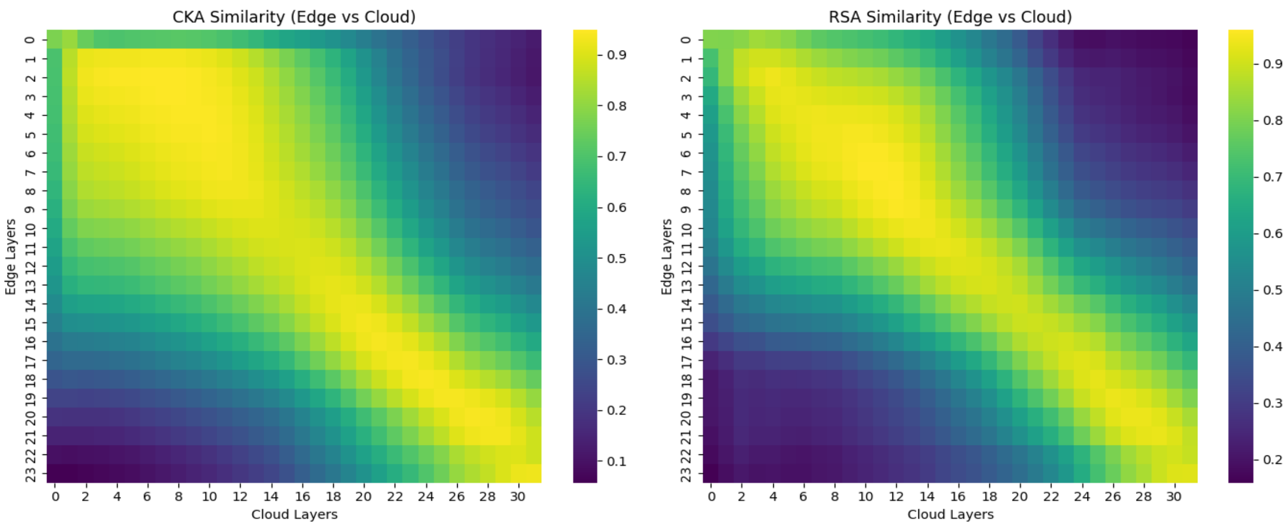


Fig. 5: Heatmaps of structural similarity between edge and cloud model layers. (**Left**) Centered Kernel Alignment (CKA) similarity scores. (**Right**) Representational Similarity Analysis (RSA) scores. Brighter areas indicate higher similarity.

structural similarity. Given two layer representation matrices R and $R^{'} \in \mathbb{R}^{N\times D}$, the computation is expressed as [28]:

$$HSIC\left(S_e^{(l_e)}, S_c^{(l_c)}\right) = \frac{1}{(N-1)^2} tr\left(H_N S_e^{(l_e)} H_N S_c^{(l_c)}\right) \quad s.t. \quad H_N = I_N - \frac{1}{N} 1_N 1_N^T \tag{12}$$

where $S_c^{(l_c)} = O_c^{(l_c)} O_c^T$ denotes the kernel similarity matrix of the $l_c - th$ layer in the cloud model, computed from its output representation matrix $O_c^{(l_c)}$. $S_e^{\left(l^{(l_e)}\right)} = RR'^T$ denotes the kernel similarity matrix of the edge model at layer l_e. H_N is the centering matrix, which is used to center both kernel matrices, enabling the evaluation of their structural alignment.

Normalization is a critical step that endows the similarity measure with a set of desirable mathematical properties. Specifically, it is expressed as [28]:

$$CKA\left(O_e^{(l_e)}, O_c^{(l_c)}\right) = \frac{HSIC\left(S_e^{(l_e)}, S_c^{(l_c)}\right)}{\sqrt{HSIC\left(S_e^{(l_e)}, S_e^{(l_e)}\right) \cdot HSIC\left(S_c^{(l_c)}, S_c^{(l_c)}\right)}} \tag{13}$$

where $m_{CKA}\left(O_e^{(l_e)}, O_c^{(l_c)}\right) \in [0,1]$ [29].

We also introduce a constraint on attention behavior consistency. Specifically, in autoregressive language models, a strict lower triangular attention mask is applied at each layer to ensure that each token can only attend to preceding positions. Cosine similarity is first used to compute the pairwise similarity between all samples within the same layer ll of the SLM, as defined in [29], and is given by:

$$S_{RSA,e}^{(l_e)} = \frac{O_{e,i}^{(l_e)} \cdot O_{e,j}^{(l_e)}}{\left\|O_{e,i}^{(l_e)}\right\| \left\|O_{e,j}^{(l_e)}\right\|} \tag{14}$$

Similarly, $S_{RSA,c}^{(l_c)}$ can be obtained.

Subsequently, the lower triangular part of each similarity matrix is extracted and flattened into vectors $s_{RSA,e}^{(l_e)}$ and $s_{RSA,c}^{(l_c)}$. The Pearson correlation coefficient between these two vectors is then computed to quantify their alignment. So,

$$RSA\left(O_e^{(l_e)}, O_c^{(l_c)}\right) = \text{Corr}\left(s_{RSA,e}^{(l_e)}, s_{RSA,c}^{(l_c)}\right) \tag{15}$$

Given predefined similarity thresholds θ_{CKA} and θ_{RSA}, we compute the similarity between a target layer $l_e \in \{1, 2, \ldots, n\}$ in the edge model and all candidate layers $l_c \in \{1, 2, \ldots, N\}$ in the cloud model, in order to evaluate their structural and semantic alignment. Subsequently, the most structurally aligned cloud layer l_c^* with respect to l_e is selected such that it satisfies the following threshold condition:

$$\begin{aligned} l_c^* &= \arg\max s\left(O_e^{(l_e)}, O_c^{(l_c)}\right) \\ s.t. \quad m_{CKA}&\left(O_e^{(l_e)}, O_c^{(l_c)}\right) \geq \theta_{CKA} \\ RSA&\left(O_e^{(l_e)}, O_c^{(l_c)}\right) \geq \theta_{RSA} \end{aligned} \tag{16}$$

where $s(\cdot,\cdot)$ denotes a general similarity function between the layer representations.

During inference, shallow layers tend to capture fundamental linguistic features such as grammar and syntax. These features are critical for downstream semantic understanding and are thus more sensitive to information loss. In contrast, deeper layers predominantly encode semantic details and abstract contextual features, which are relatively less stringent in their error tolerance. As shown in Fig.5, we visualize the structural similarity matrices between edge and cloud models using both CKA and RSA metrics. Brighter regions indicate higher layer-wise similarity. Notably, the similarity maps exhibit a clear diagonal trend, suggesting a strong layer-wise alignment across the two heterogeneous models. Based on these observations, we define our layer sharing strategy: among candidate pairs that satisfy the similarity threshold, we prefer to select shallower layers for sharing to preserve core semantic representations. The final set of KV cache sharing layers is defined as $L_{Shared} = \{L_1, L_2, \cdots, L_n\}$. It is worth

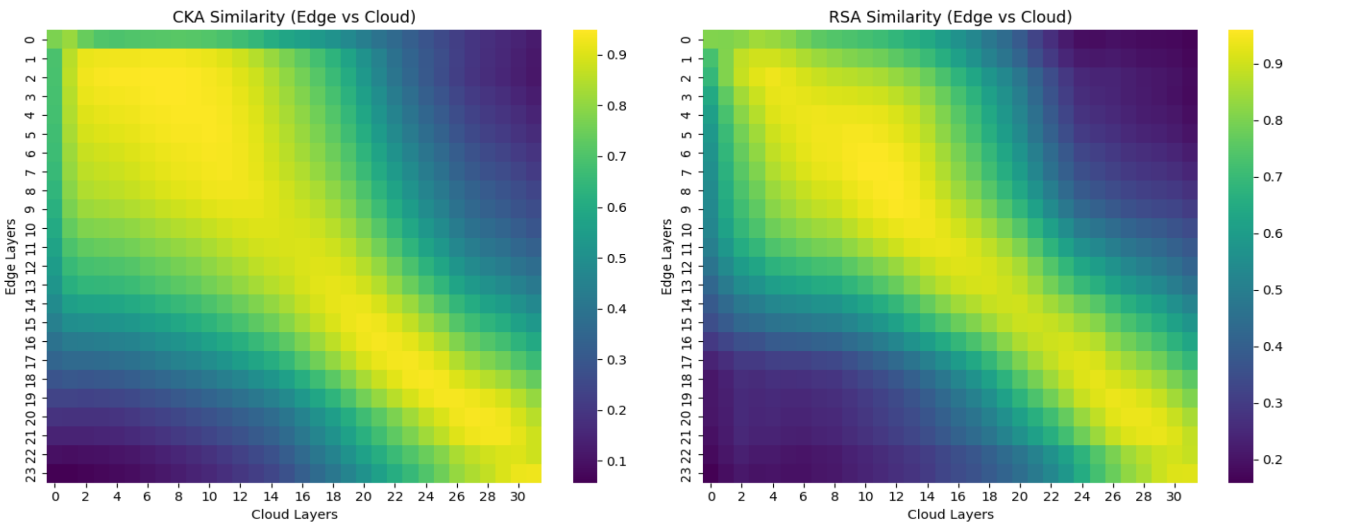


图 5：边缘模型层和云模型层之间结构相似性的热图。（左）居中核对齐 (CKA) 相似度得分。（右）表征相似性分析 (RSA) 分数。较亮的区域表示相似度较高。

结构相似性。给定两层表示矩阵R和$R' \in \mathbb{R}^{N\times D}$，计算表示为[28]:

$$HSIC\left(S_e^{(l_e)}, S_c^{(l_c)}\right) = \frac{1}{(N-1)^2} tr\left(H_N S_e^{(l_e)} H_N S_c^{(l_c)}\right) \\ s.t.\quad H_N = I_N - \frac{1}{N} 1_N 1_N^T \tag{12}$$

其中 $S_c^{(l_c)} = O_c^{(l_c)} O_c^T$ 表示云模型中 $l_c - th$ 层的内核相似度矩阵，根据其输出表示矩阵 $O_c^{(l_c)}$ 计算得出。$S_e^{\left(l^{(l_e)}\right)} = RR'^T$表示$l_e$层边缘模型的核相似度矩阵。$H_N$ 是居中矩阵，用于将两个核矩阵居中，从而能够评估它们的结构对齐。

归一化是一个关键步骤，它赋予相似性度量一组理想的数学属性。具体可表示为[28]:

$$CKA\left(O_e^{(l_e)}, O_c^{(l_c)}\right) = \frac{HSIC\left(S_e^{(l_e)}, S_c^{(l_c)}\right)}{\sqrt{HSIC\left(S_e^{(l_e)}, S_e^{(l_e)}\right) \cdot HSIC\left(S_c^{(l_c)}, S_c^{(l_c)}\right)}} \tag{13}$$

其中 $m_{CKA}\left(O_e^{(l_e)}, O_c^{(l_c)}\right) \in [0, 1]$ [29]。

我们还引入了对注意力行为一致性的约束。具体来说，在自回归语言模型中，在每一层应用严格的下三角注意掩模，以确保每个标记只能关注前面的位置。余弦相似度首先用于计算 SLM 同一层 ll 内所有样本之间的成对相似度，如[29]中所定义，由下式给出:

$$S_{RSA,e}^{(l_e)} = \frac{O_{e,i}^{(l_e)} \cdot O_{e,j}^{(l_e)}}{\left\|O_{e,i}^{(l_e)}\right\| \left\|O_{e,j}^{(l_e)}\right\|} \tag{14}$$

同理可得$S_{RSA,c}^{(l_c)}$。

随后，提取每个相似度矩阵的下三角部分并将其展平为向量$s_{RSA,e}^{(l_e)}$和$s_{RSA,c}^{(l_c)}$。然后计算这两个向量之间的皮尔逊相关系数来量化它们的对齐情况。所以，

$$RSA\left(O_e^{(l_e)}, O_c^{(l_c)}\right) = \text{Corr}\left(s_{RSA,e}^{(l_e)}, s_{RSA,c}^{(l_c)}\right) \tag{15}$$

给定预定义的相似度阈值θ_{CKA}和θ_{RSA}，我们计算边缘模型中的目标层$l_e \in \{1, 2, \ldots, n\}$与云模型中的所有候选层$l_c \in \{1, 2, \ldots, N\}$之间的相似度，以评估它们的结构和语义对齐。随后，选择相对于 l_e 结构最一致的云层 l_c^*，使其满足以下阈值条件:

$$l_c^* = \arg\max s\left(O_e^{(l_e)}, O_c^{(l_c)}\right) \\ s.t.\quad m_{CKA}\left(O_e^{(l_e)}, O_c^{(l_c)}\right) \geq \theta_{CKA} \\ RSA\left(O_e^{(l_e)}, O_c^{(l_c)}\right) \geq \theta_{RSA} \tag{16}$$

其中 $s(\cdot, \cdot)$ 表示层表示之间的一般相似性函数。

在推理过程中，浅层倾向于捕获基本的语言特征，例如语法和句法。这些特征对于下游语义理解至关重要，因此对信息丢失更加敏感。相比之下，更深的层主要编码语义细节和抽象上下文特征，其容错性相对不太严格。如图 5 所示，我们使用 CKA 和 RSA 指标可视化边缘模型和云模型之间的结构相似性矩阵。较亮的区域表示较高的层间相似性。值得注意的是，相似性图表现出明显的对角线趋势，表明两个异构模型之间存在很强的分层对齐。基于这些观察，我们定义了层共享策略：在满足相似性阈值的候选对中，我们更喜欢选择较浅的层进行共享以保留核心语义表示。最终的 KV 缓存共享层集合定义为 $L_{Shared} = \{L_1, L_2, \cdots, L_n\}$。值得

noting that if the SLMs deployed within a region share the same architecture, shallow-layer KV caches can be directly shared across nodes without the need for structural alignment or layer matching.

B. Dimensionality Reduction via think

To further reduce the computational and transmission overhead of attention heads during collaborative inference, we introduce a dimensionality reduction strategy inspired by the pruning objective proposed in [30]. Specifically, for each attention head ii, we aim to select a subset of channels that preserve the main interaction patterns between queries and keys. The pruning objective is formulated as [30]:

$$\begin{aligned} &\min_{S} \left\| Q_i K_i^T - Q_i S \left(k_i S\right)^T \right\|_F \\ &s.t.\ \ trace\left(S\right) = \lfloor \left(1-\lambda\right) D \rfloor \end{aligned} \tag{17}$$

where $Q_i \in \mathbb{R}^{S\times D}$ and $K_i \in \mathbb{R}^{S\times D}$ denote the query and key matrices of the $i-th$ attention head, with sequence length S and head dimension D. $S \in \{0,1\}^{D\times D}$ is a binary diagonal matrix, indicating whether a given channel (dimension) is retained 1 or pruned 0. $\lambda \in [0,1]$ controls the pruning ratio, such that only $(1-\lambda)\,D$ channels are preserved, and $\|\cdot\|_F$ is the Frobenius norm.

When the dimensionality of KV cache vectors is reduced from d_c to d_e, we adopt the estimation formula proposed in prior work [31] to quantify the reduction in computational cost and transmission overhead. Specifically, the reductions in computational FLOPs and I/O transmission volume are expressed as:

$$\begin{aligned} \triangle_{FLOPs} &= L \cdot 8bmk\left(d_c - d_e\right) \\ \triangle_{I/O} &= L \cdot \left(4bmk\left(d_c - d_e\right) + 4bk\left(d_c - d_e\right)\right) \end{aligned} \tag{18}$$

where b, m, k, d, and $h = k \cdot d$ represent the batch size, sequence length, number of attention heads, hidden size of each head, and overall hidden size, respectively.

Assuming model parameters$b = 1, m = 1024, k = 32, d_c = 80, d_e = 64$,and $L = 32$, the changes in computational cost and data transmission requirements are calculated as $\triangle_{FLOPs} = 134217728 FLOPs$, and $\triangle_{I/O} = 66.9MB$. Under the conditions of an edge device with a floating-point computational capacity of 100 GFLOPs and a communication bandwidth of 10 Mbps, dimensionality reduction reduces communication latency by 6.69 seconds and inference time by approximately 1.34 milliseconds. These results demonstrate that optimizing the KV cache significantly reduces communication costs between the cloud and edge, as well as the computational burden of inference on edge devices.

C. Communication total optimization

As a key enabling technology in 6G, terahertz communication offers ultra-high bandwidth for edge devices. However, its extremely short wavelength and limited penetration capacity significantly constrain the effective transmission range, making multi-hop relaying infeasible in many scenarios. As a result, direct transmission between the cloud and edge or among edge devices is preferred to minimize the latency caused by link hopping.

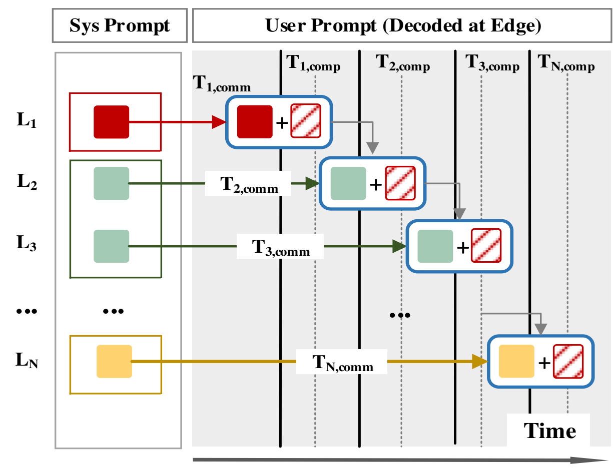


Fig. 6: Pipeline Parallel Optimization Strategy. The horizontal axis represents time, while the vertical axis depicts the hierarchical structure of the model.

According to the layer-wise similarity analysis (see Eq. 16), the deeper layers of the edge SLMs are generally more structurally aligned with those of the cloud LLM. As a result, the edge SLM prioritizes reusing deep-layer contextual KV caches generated by the cloud, with dimensionality reduction applied when necessary to enhance its capability for long-range context understanding. In addition, CE-LSLM also supports cache sharing among regional edge devices. When one node completes the KV cache computation for the shallow layers of the system prompt ahead of others, the results can be transmitted over the local interconnect to accelerate the user prompt processing of other local SLMs. Let M denote the total number of layers in SLM, and n represent the number of layers whose contextual caches are reused from LLM. We define the cache source function $src\left(l\right)$ for the $l-th$ layer as:

$$src\left(l\right) = \begin{cases} arg\min_{\eta \in \{local, peer\}} cost_\eta\left(l\right), & l < n \\ cloud, & l \geq n \end{cases} \tag{19}$$

where $cost_\eta\left(l\right)$ represents the cost of obtaining the cache for layer l from source η.

To alleviate inference latency caused by sequential execution, a pipelined scheduling mechanism is introduced to enable the parallel coordination between attention computation and contextual KV cache transmission. In traditional processing paradigms, the edge SLM must first complete the KV cache computation for the entire context (e.g., system prompt) before user prompt decoding can be initiated. In contrast, under the proposed strategy, the edge SLM begins computing the contextual KV caches layer by layer from the shallow layers (layer 1 to $M-n$), which are either computed locally or reused from nearby peers. Meanwhile, the deep-layer caches

值得注意的是，如果一个区域内部署的SLM共享相同的架构，浅层KV缓存可以直接在节点之间共享，而不需要结构对齐或层匹配。

B. Dimensionality Reduction via think

为了进一步减少协作推理过程中注意力头的计算和传输开销，我们引入了一种受[30]中提出的剪枝目标启发的降维策略。具体来说，对于每个注意力头 ii，我们的目标是选择保留查询和键之间主要交互模式的通道子集。剪枝目标表述为[30]：

$$\begin{aligned} &\min_{S} \left\| Q_i K_i^T - Q_i S \left(k_i S\right)^T \right\|_F \\ &s.t. \quad trace\left(S\right) = \lfloor \left(1-\lambda\right) D \rfloor \end{aligned} \tag{17}$$

其中$Q_i \in \mathbb{R}^{S\times D}$和$K_i \in \mathbb{R}^{S\times D}$表示$i-th$注意头的查询矩阵和键矩阵，序列长度为$S$，头维度为$D$。$S \in \{0,1\}^{D\times D}$是一个二元对角矩阵，表示给定通道（维度）是保留 1 还是剪枝 0。$\lambda \in [0,1]$ 控制剪枝比率，使得仅保留 $(1-\lambda)$ D 个通道，而 $\|\cdot\|_F$ 是 Frobenius 范数。

当KV缓存向量的维数从d_c减少到d_e时，我们采用先前工作[31]中提出的估计公式来量化计算成本和传输开销的减少。具体来说，计算 FLOP 和 I/O 传输量的减少表示为：

$$\begin{aligned} \triangle_{FLOPs} &= L \cdot 8bmk\left(d_c - d_e\right) \\ \triangle_{I/O} &= L \cdot \left(4bmk\left(d_c - d_e\right) + 4bk\left(d_c - d_e\right)\right) \end{aligned} \tag{18}$$

其中b, m, k, d和$h = k \cdot d$分别表示批量大小、序列长度、注意力头数量、每个头的隐藏大小和总体隐藏大小。

假设模型参数$b=1, m=1024, k=32, d_c=80, d_e=64$和$L=32$，计算成本和数据传输要求的变化计算为$\triangle_{FLOPs}=134217728FLOPs$和$\triangle_{I/O}=66.9MB$。在边缘设备浮点计算能力为100 GFLOPs、通信带宽为10 Mbps的条件下，降维可将通信延迟减少6.69秒，推理时间减少约1.34毫秒。这些结果表明，优化 KV 缓存可以显着降低云端和边缘之间的通信成本，以及边缘设备上推理的计算负担。

C. Communication total optimization

作为 6G 的关键使能技术，太赫兹通信为边缘设备提供超高带宽。然而，其极短的波长和有限的穿透能力极大地限制了有效传输范围，使得多跳中继在许多场景下不可行。云与边、边与边之间直接传输

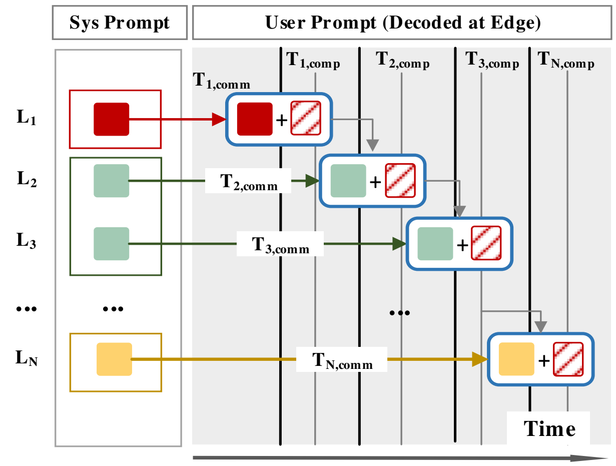


图 6：管道并行优化策略。横轴代表时间，纵轴描绘模型的层次结构。

优选设备以最小化由链路跳变引起的延迟。

根据逐层相似性分析（参见方程 16），边缘 SLM 的较深层通常在结构上与云 LLM 的结构更加一致。因此，边缘SLM优先重用云生成的深层上下文KV缓存，并在必要时应用降维以增强其远程上下文理解的能力。此外，CE-LSLM还支持区域边缘设备之间的缓存共享。当一个节点先于其他节点完成系统提示浅层的KV缓存计算时，结果可以通过本地互连传输，以加速其他本地SLM的用户提示处理。令 M 表示 SLM 中的总层数，n 表示从 LLM 重用上下文缓存的层数。我们将 $l-th$ 层的缓存源函数 $src\left(l\right)$ 定义为：

$$src\left(l\right) = \begin{cases} arg\min_{\eta\in\{local,peer\}} cost_\eta\left(l\right), & l < n \\ cloud, & l \geq n \end{cases} \tag{19}$$

其中 $cost_\eta\left(l\right)$ 表示从源 η 获取层 l 的缓存的成本。

为了减轻顺序执行引起的推理延迟，引入了流水线调度机制，以实现注意力计算和上下文 KV 缓存传输之间的并行协调。在传统的处理范例中，边缘SLM必须首先完成整个上下文（例如系统提示）的KV缓存计算，然后才能启动用户提示解码。相比之下，在所提出的策略下，边缘SLM开始从浅层（第1层到$M-n$）逐层计算上下文KV缓存，这些缓存要么在本地计算，要么从附近的对等点重用。同时，深层缓存

(layer $M-n+1$ to M) are concurrently loaded from the cloud. These two computation and transmission paths converge at the intermediate layer, allowing user prompt inference to begin even before all context caches are fully prepared, thereby significantly reducing the overall response latency. This parallel process can be formulated as follows:

$$T_{pip}^{(l)} = \begin{cases} \max\left\{t_{comm}^{(l)}\left(src\left(l\right)\right), t_{comp}^{(l-1)}\right\}, & l \in [1, M-n] \\ \max\left\{t_{comm}^{(l)}\left(cloud\right), t_{comp}^{(l-1)}\right\}, & l \in [M-n+1, M] \end{cases} \tag{20}$$

where $t_{comm}^{(l)}(\cdot)$ denotes the communication time required to load the KV cache for layer l, and $t_{comp}^{(l-1)}$ represents the computation time of the previous layer.

Fig. 6 visually illustrates the parallel overlapping process of computation and transmission times across layers during the pipelined procedure. The solid vertical lines represent the time consumed for transmitting each layer's cloud-side cache to the edge device, while the gray dashed lines indicate the time required for the edge device to compute the user prompt. Since the computation of the previous layer $l-1$ and the transmission of the next layer l are performed in parallel, the overlapping portion of computation time is covered, and only the larger value between the computation and transmission time is retained.

VI. Experiments

This section presents a comprehensive evaluation of the proposed CE-LSLM collaborative inference framework. We compare three LLM deployment strategies: cloud-only, edge-only, and cloud-edge collaboration, focusing on key performance metrics such as text generation fidelity, inference latency.

A. Experimental Setup

Opt-2-6.7B is deployed as the cloud-based LLM, responsible for long-context inference, while Opt-2-1.3B serves as the edge-side SLM, performing local inference. OPT, an open-source large language model developed by Meta, adopts a decoder-only causal architecture and supports up to 2048 positional tokens, making it well-suited for autoregressive natural language processing tasks. We extract both short- and long-sequence experimental data from the ShareGPT_V3 dataset, which contains extensive real-world user interactions with ChatGPT across various scenarios. Additionally, we utilize the XSum dataset, which comprises BBC articles and serves as a benchmark for evaluating the text generation quality of LLMs. Regarding hardware selection, we employ the NVIDIA A800, a high-performance cloud computing device commonly used in both cloud and edge computing environments. The detailed hardware specifications are presented in Table I.

While vLLM-based inference services exhibit strong asynchronous scheduling and high throughput in practical deployments, our lab environment employs shared GPU resources with limited memory capacity. This setup reflects typical deployment constraints such as multi-tenant execution and

TABLE I: Specifications of GPU Used in Our Experiments.

Symbols	Memory	Mem.Band	FP16 Peak F.
NVIDIA A800 80GB PCIe	80 GB	2030 GB/s	77.9 TFLOPs

edge-based inference nodes. Given these conditions, our experimental comparisons are conducted under controlled assumptions to highlight the performance characteristics of different deployment strategies, rather than to fully replicate production environments. The three deployment strategies are explained as follows:

- **Cloud-only**: all inference tasks are executed in the cloud. This setting includes both a naive baseline that recomputes the system prompt for each query without cache reuse, and an optimized vLLM-based variant that precomputes and caches system prompt KV to accelerate batched inference [11], [12].
- **Edge-only**: all computations are performed entirely at the edge, without any collaborative inference or acceleration mechanisms. This serves as a baseline to evaluate the inference performance of edge devices under non-collaborative conditions.
- **Cloud-Edge** (ours): context processing and user prompt inference are performed separately on cloud and edge models. By jointly retrieving system prompt KV caches from both cloud and edge, the edge model is enabled to efficiently handle multiple user requests in parallel, supporting fast batched inference.

The primary performance evaluation metrics include: (1) End-to-end transmission time: the total time required to complete the entire inference task, including the cloud-based LLM computation time, communication transmission time, and the edge-based SLM computation time. (2) User-perceived latency, defined as the average time from request initiation to receipt of the complete response, serving as a measure of actual user experience; (3) Normalized latency (ms/token), representing the average inference time per generated token; (4) Time to First Token (TTFT), measuring the time elapsed from request submission to the generation of the first token; and (5) Generation quality, evaluated by a weighted combination of BERTScore and ChatGPT-based scoring, where higher values indicate better semantic alignment between generated and reference texts. All experiments are repeated five times and averaged across batches to obtain stable single-inference performance measurements.

B. Performance in Static Request Scenarios

We selected 100 samples from each of the XSum [32] and MMLU [33] datasets and modified them accordingly to suit the task scenarios within our request-response framework. Specifically, XSum article texts were used as contextual inputs, and task-specific queries were generated based on different topics. For MMLU, background information was added to enrich the prompt, and the original multiple-choice format was reformulated into open-ended question answering. The adapted inputs range from approximately 300 to 500 tokens in length

(层$M-n+1$到M)同时从云端加载。这两条计算和传输路径在中间层汇聚，允许用户在所有上下文缓存完全准备好之前就开始即时推理，从而显着降低整体响应延迟。这个并行过程可以表述如下:

$$T_{pip}^{(l)}=\begin{cases}\max\left\{t_{comm}^{(l)}\left(src\left(l\right)\right),t_{comp}^{(l-1)}\right\}, & l\in[1,M-n]\\ \max\left\{t_{comm}^{(l)}\left(cloud\right),t_{comp}^{(l-1)}\right\}, & l\in[M-n+1,M]\end{cases} \tag{20}$$

其中$t_{comm}^{(l)}\left(\cdot\right)$表示加载第$l$层的KV缓存所需的通信时间，$t_{comp}^{(l-1)}$表示上一层的计算时间。

图6直观地说明了流水线过程中跨层计算和传输时间的并行重叠过程。垂直实线表示将各层云端缓存传输到边缘设备所消耗的时间，灰色虚线表示边缘设备计算用户提示所需的时间。由于上一层$l-1$的计算和下一层l的传输是并行执行的，因此覆盖了计算时间的重叠部分，并且仅保留计算时间和传输时间之间的较大值。

六. 实验

本节对所提出的 CE-LSLM 协作推理框架进行全面评估。我们比较了三种 LLM 部署策略：仅云、仅边缘和云边缘协作，重点关注文本生成保真度、推理延迟等关键性能指标。

A. Experimental Setup

Opt-2-6.7B被部署为基于云的LLM，负责长上下文推理，而Opt-2-1.3B作为边缘侧SLM，执行本地推理。 OPT是Meta开发的开源大语言模型，采用仅解码器的因果架构，最多支持2048个位置标记，非常适合自回归自然语言处理任务。我们从 ShareGPT V3 数据集中提取短序列和长序列实验数据，其中包含跨各种场景与 ChatGPT 的广泛真实用户交互。此外，我们还利用 XSum 数据集，该数据集包含 BBC 文章，并作为评估大语言模型文本生成质量的基准。在硬件选择方面，我们采用了NVIDIA A800，这是一款常用于云和边缘计算环境的高性能云计算设备。详细的硬件规格如表 I 所示。

虽然基于 vLLM 的推理服务在实际部署中表现出强大的异步调度和高吞吐量，但我们的实验室环境使用内存容量有限的共享 GPU 资源。此设置反映了典型的部署约束，例如多租户执行和

表 I：我们实验中使用的 GPU 规格。

Symbols	Memory	Mem.Band	FP16 Peak F.
NVIDIA A800 80GB PCIe	80 GB	2030 GB/s	77.9 TFLOPs

基于边缘的推理节点。考虑到这些条件，我们的实验比较是在受控假设下进行的，以突出不同部署策略的性能特征，而不是完全复制生产环境。这三种部署策略解释如下:

- 仅云：所有推理任务都在云端执行。此设置包括为每个查询重新计算系统提示而不重用缓存的朴素基线，以及预先计算和缓存系统提示 KV 以加速批量推理的基于 vLLM 的优化变体 [11]、[12]。
- 仅边缘：所有计算完全在边缘执行，没有任何协作推理或加速机制。这可以作为评估非协作条件下边缘设备的推理性能的基线。
- 云-边缘（我们的）：上下文处理和用户提示推理分别在云和边缘模型上执行。通过从云端和边缘联合检索系统提示 KV 缓存，使边缘模型能够高效地并行处理多个用户请求，支持快速批量推理。

主要性能评估指标包括：（1）端到端传输时间：完成整个推理任务所需的总时间，包括基于云的LLM计算时间、通信传输时间和基于边缘的SLM计算时间。（2）用户感知时延，定义为从发起请求到收到完整响应的平均时间，作为实际用户体验的衡量标准；(3) 归一化延迟（ms/token），表示每个生成的token的平均推理时间；（4）Time to First Token（TTFT），测量从请求提交到第一个令牌生成所经过的时间；(5) 生成质量，通过 BERTScore 和基于 ChatGPT 的评分的加权组合进行评估，其中较高的值表示生成文本和参考文本之间更好的语义对齐。所有实验重复五次并在批次之间取平均值，以获得稳定的单推理性能测量。

B. Performance in Static Request Scenarios

我们从 XSum [32] 和 MMLU [33] 数据集中选择了 100 个样本，并相应地修改它们以适应我们的请求响应框架内的任务场景。具体来说，XSum 文章文本用作上下文输入，并根据不同主题生成特定于任务的查询。对于MMLU，增加了背景信息以丰富提示，并将原来的多项选择题格式重新表述为开放式问答。调整后的输入长度范围约为 300 到 500 个令牌

TABLE II: Evaluation of Inference Efficiency and Resource Overhead under Different Deployment Strategies

Dataset	Deployment stragety		TTFT(ms)	Total Time Cost (s)	Base Memory Usage (MB)	Inference Memory (MB)	User Data Upload Ratio(%)	Transmitted Data Size (MB)	Score
xsum	Cloud-only	Naïve-cloud	652.12	519.63	12700.03	8.202	100	0.33	0.9025
		vLLM-ra	304.25	78.72	13781.41	44075.00	100	0.33	0.896
	Edge-only	Naïve-edge	311.592	384.34	2509.61	8.13	0	0	0.7009
	Cloud-Edge	CE-LSLM	210.24	68.58	2941.86	2756	0	0	0.877
mmlu	Cloud-only	Naïve-cloud	412.99	199.11	12700.03	8.13	100	0.31	0.891
		vLLM-ra	335.59	26.01	13781.41	55488.00	100	0.31	0.873
	Edge-only	Naïve-edge	61.27	148.34	2509.61	8.13	0	0	0.72
	Cloud-Edge	CE-LSLM	170.03	11.00	2941.86	2756	0	0	0.854

and reflect medium-complexity inference and comprehension scenarios.

In the Cloud-only strategy, both the system prompt and user queries are required to be fully transmitted to the cloud for inference. Although the computational overhead associated with repeated processing of system prompts can be mitigated using caching mechanisms such as Relay Attention [11], all requests must still be centrally processed on the cloud. This leads to increased communication overhead and raises potential privacy concerns due to the transmission of sensitive user data.

In contrast, under the proposed CE-LSLM strategy, sensitive user content is entirely processed on the edge device, with only deep-layer KV caches containing general knowledge being fetched from the cloud. For multi-batch inference tasks, the KV cache can be transmitted once to the edge and locally reused, thereby significantly reducing communication latency. In addition, model computation and cache loading are executed in parallel, eliminating extra waiting time. As shown in Table 2, CE-LSLM demonstrates a significant advantage in terms of Time to First Token (TTFT). In terms of generation quality, the BERTScore results are comparable to those of the Cloud-only strategy, indicating that the KV cache sharing mechanism enhances inference efficiency without compromising semantic consistency or output fidelity.

Under scenarios with concurrent requests or long-context inputs, the storage of intermediate states can significantly consume GPU memory. As shown in the table II, the peak memory requirement for vLLM-ra reaches up to $44GB$, which clearly exceeds the resource capacity of most edge devices. To improve request-handling capability, the Naive Edge-only strategy typically resorts to truncating the input context, thereby weakening semantic guidance and contextual grounding, which ultimately degrades the quality of the generated text. Although edge models can deliver low response latency for lightweight tasks, the absence of parallel scheduling and cache reuse mechanisms leads to substantial inference delays when handling complex tasks. By comparison, CE-LSLM effectively mitigates the above resource bottlenecks through cache reuse strategies, enhancing both generation quality and the responsiveness of edge deployments.

The CE-COLLM framework incorporates an early exit mechanism, enabling partial inference to be completed at the edge without requiring cloud processing [16]. However, this mechanism relies on a predefined confidence threshold if the confidence score of the generated output is low, the intermediate results will be transmitted to the cloud for further computation. Additionally, a stable network connection is required to prevent increased inference latency or task failure.

C. System Stability under High Request Rates

In this experiment, the ShareGPT_V3 dataset was used to simulate a high-concurrency request environment, aiming to evaluate system performance in terms of response latency and normalized latency under varying load intensities. A time-window-based scheduling strategy was employed to enhance controllability and measurement accuracy. LLM was configured to concurrently serve inference requests from five SLMs, covering both resource-constrained and resource-sufficient scenarios. Different prefix lengths were considered, and the maximum batch sizes supported by the memory capacity of both cloud and edge models were assigned to reflect memory-induced performance bottlenecks. It should be noted that the Naive Cloud-only and Naive Edge-only configurations were excluded from the comparison, as they fail to support sustained batch processing under resource constraints and frequently trigger memory overflows. The Naive Edge setup additionally suffers from unstable generation quality.

Fig. 7 presents the inference latency results for cloud-based inference using vLLM and the proposed CE-LSLM under sustained high-concurrency request scenarios. Due to the deployment of SLMs at the edge, CE-LSLM exhibits real-time responsiveness. In lightweight input scenarios, CE-LSLM maintains an average inference latency of approximately 1 second across varying request rates, demonstrating high stability. In contrast, the LLM executes a greater number of

表二：不同部署策略下的推理效率和资源开销评估

Dataset	Deployment stragety		TTFT(ms)	Total Time Cost (s)	Base Memory Usage (MB)	Inference Memory (MB)	User Data Upload Ratio(%)	Transmitted Data Size (MB)	Score
xsum	Cloud-only	Naïve-cloud	652.12	519.63	12700.03	8.202	100	0.33	0.9025
		vLLM-ra	304.25	78.72	13781.41	44075.00	100	0.33	0.896
	Edge-only	Naïve-edge	311.592	384.34	2509.61	8.13	0	0	0.7009
	Cloud-Edge	CE-LSLM	210.24	68.58	2941.86	2756	0	0	0.877
mmlu	Cloud-only	Naïve-cloud	412.99	199.11	12700.03	8.13	100	0.31	0.891
		vLLM-ra	335.59	26.01	13781.41	55488.00	100	0.31	0.873
	Edge-only	Naïve-edge	61.27	148.34	2509.61	8.13	0	0	0.72
	Cloud-Edge	CE-LSLM	170.03	11.00	2941.86	2756	0	0	0.854

并反映中等复杂度的推理和理解场景。

在Cloud-only策略中，系统提示和用户查询都需要完全传输到云端进行推理。尽管可以使用 Relay Attention [11] 等缓存机制来减轻与重复处理系统提示相关的计算开销，但所有请求仍然必须在云上集中处理。这会导致通信开销增加，并由于敏感用户数据的传输而引发潜在的隐私问题。

相比之下，在所提出的CE-LSLM策略下，敏感的用户内容完全在边缘设备上处理，只有包含一般知识的深层KV缓存从云端获取。对于多批次推理任务，KV缓存可以一次传输到边缘并在本地重复使用，从而显着减少通信延迟。此外，模型计算和缓存加载是并行执行的，消除了额外的等待时间。如表 2 所示，CE-LSLM 在首次令牌时间 (TTFT) 方面表现出显着优势。在生成质量方面，BERTScore 结果与 Cloud-only 策略相当，表明 KV 缓存共享机制在不影响语义一致性或输出保真度的情况下提高了推理效率。

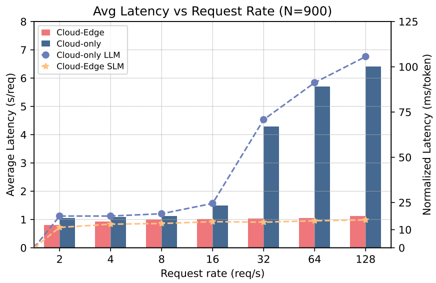
在并发请求或长上下文输入的场景下，中间状态的存储会显着消耗GPU内存。如表二所示，vLLM-ra的峰值内存需求高达44GB，这明显超出了大多数边缘设备的资源容量。为了提高请求处理能力，仅 Naive Edge 策略通常会采取截断输入上下文的方式，从而削弱语义指导和上下文基础，最终降低生成文本的质量。尽管边缘模型可以为轻量级任务提供较低的响应延迟，但缺乏并行调度和缓存重用机制会导致处理复杂任务时出现严重的推理延迟。相比之下，CE-LSLM 通过缓存复用策略有效缓解上述资源瓶颈，提高生成质量和边缘部署的响应能力。

CE-COLLM框架采用了提前退出机制，使得部分推理能够在边缘完成，而不需要云处理[16]。然而，该机制依赖于预定义的置信度阈值，如果生成的输出的置信度分数较低，则中间结果将传输到云端进行进一步计算。此外，还需要稳定的网络连接来防止推理延迟增加或任务失败。

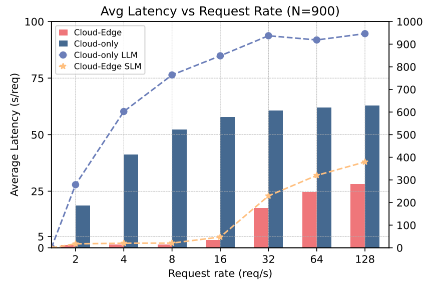
C. System Stability under High Request Rates

本实验使用ShareGPT V3数据集模拟高并发请求环境，旨在评估不同负载强度下的响应延迟和归一化延迟方面的系统性能。采用基于时间窗口的调度策略来增强可控性和测量精度。 LLM 配置为同时服务来自五个 SLM 的推理请求，涵盖资源受限和资源充足的场景。考虑不同的前缀长度，并分配云和边缘模型的内存容量支持的最大批量大小，以反映内存引起的性能瓶颈。值得注意的是，Naive Cloud-only 和 Naive Edge-only 配置被排除在比较之外，因为它们无法支持资源限制下的持续批处理，并且经常触发内存溢出。 Naive Edge 设置还存在生成质量不稳定的问题。

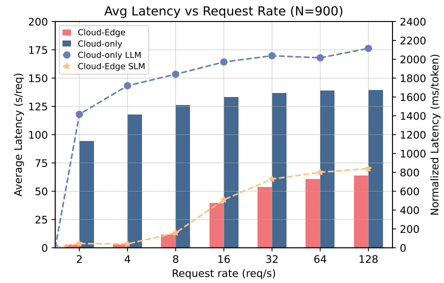
图 7 展示了在持续高并发请求场景下使用 vLLM 和提出的 CE-LSLM 进行基于云的推理的推理延迟结果。由于SLM部署在边缘，CE-LSLM表现出实时响应能力。在轻量级输入场景中，CE-LSLM 在不同的请求速率下保持大约 1 秒的平均推理延迟，表现出高稳定性。相比之下，LLM 执行的数量更多



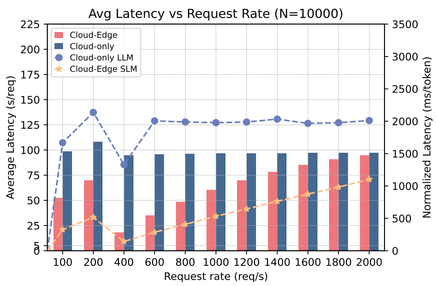
(a) System prompt length=64



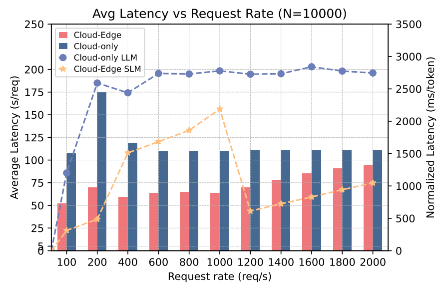
(b) System prompt length=512



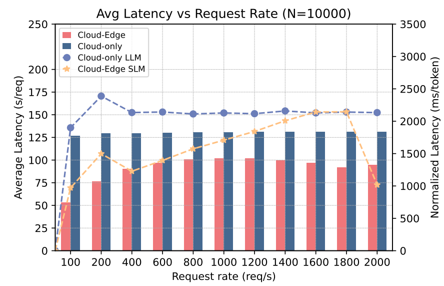
(c) System prompt length=1024



(d) System prompt length=64



(e) System prompt length=512



(f) System prompt length=1024

Fig. 7: Specifically, subfigures (a)–(c) present the latency results under resource-constrained conditions, where GPU resources are concurrently shared among multiple users. In contrast, subfigures (d)–(f) illustrate latency outcomes in resource-sufficient settings. The left y-axis denotes the average response latency, while the right y-axis indicates the normalized latency.

attention operations per layer, incurring higher overhead in memory access and activation cache usage, thereby increasing per-token inference latency. As a result, when the request rate reaches 32 $reqs$, the average latency of the Cloud-only configuration surges beyond 4 seconds—over four times the baseline latency—indicating evident queuing delays and performance degradation under moderate concurrency loads.

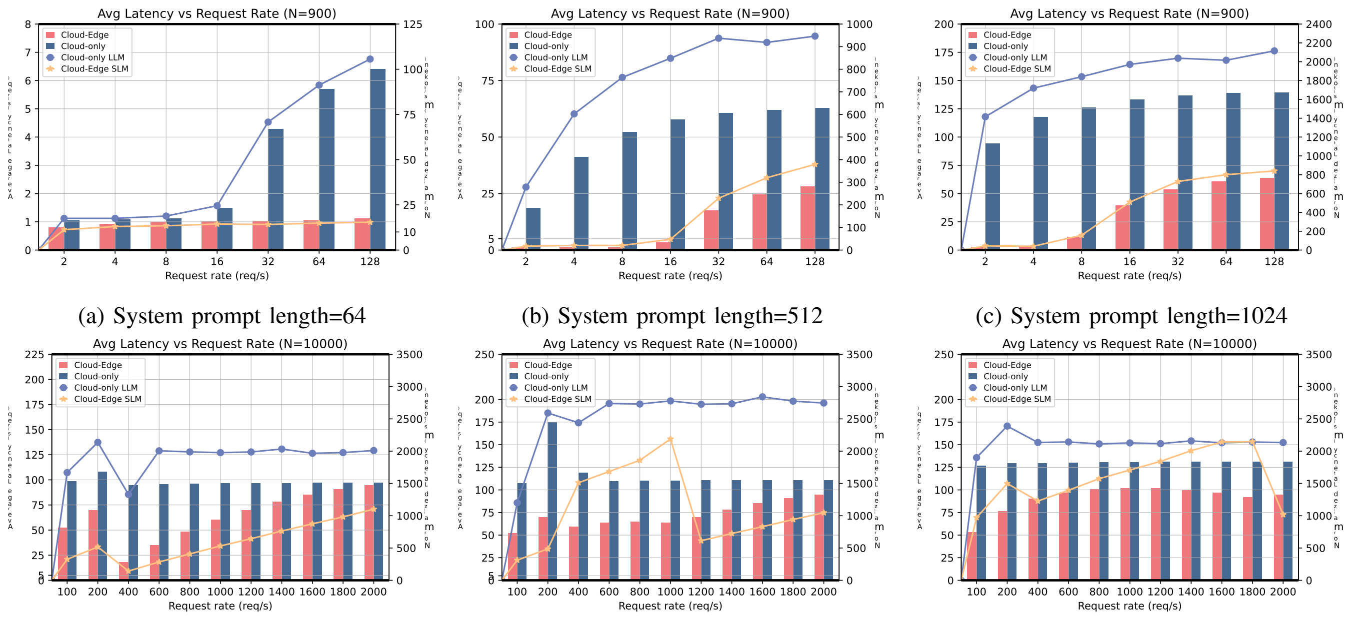
When the prefix length increases from 512 to 1024, as shown in Fig. 7(b)-(c), systems operating under the compute-only execution mode experience significant nonlinear increases in response latency under high request rates. This behavior indicates that the system bottleneck gradually shifts from being communication-bound to computation-bound due to the inability to decouple compute and memory resources. In contrast, the latency growth of CE-LSLM remains relatively stable, demonstrating greater robustness. This advantage is primarily attributed to the reuse of KV caches and the parallel coordination of model computation and memory loading, which together alleviate memory access bottlenecks and significantly improve inference stability and concurrency handling capability.

When GPU resources are sufficient, as shown in Fig. 7(d)-(f), both architectures gradually exhibit saturation behavior as the request rate increases significantly. However, CE-LSLM consistently achieves lower average latency compared to the Cloud-only configuration, indicating stronger delay control and greater system robustness under concurrent workloads and resource stress. In addition, when the prefix length exceeds 512, both average latency and normalized latency begin to show noticeable fluctuations. This behavior is attributed to the fact that small models are often used for short texts or lightweight tasks, where the shorter output sequences make per-token latency metrics more sensitive to minor temporal variations.

Overall, even under saturation conditions, the CE-LSLM architecture maintained effective task scheduling, edge cooperation, and caching mechanisms, without experiencing performance degradation due to increased network load. This reflects strong controllability of concurrent latency. By offloading a portion of the computation to the edge, the cloud-side burden was significantly reduced, enabling the system to support a larger number of edge-deployed SLM instances. In terms of throughput, at moderate request rates (e.g., 64 req/s), the CE-LSLM configuration achieved approximately 4–6× higher throughput compared to the Cloud-only baseline, thereby validating the effectiveness and rationality of edge-side role assignment in the collaborative inference architecture.

VII. Conclusion

In the process of extending generative AI services to wireless edge networks, limited memory and computational capacity have emerged as critical bottlenecks in the deployment of LLM. To address this challenge, a collaborative inference architecture was proposed, integrating LLM with SLMs. A novel service mechanism was explored in the context of 6G networks, aiming to jointly optimize generative processing and communication. The proposed architecture incorporates hierarchical cache sharing and a pipelined compute-load parallelism scheme, enabling coordinated optimization between inference execution and wireless communication resources. Furthermore, in scenarios where the cloud is unavailable or network connections are temporarily disrupted, partial inference tasks can still be executed by edge models through



(a) System prompt length=64 (b) System prompt length=512 (c) System prompt length=1024

(d) System prompt length=64 (e) System prompt length=512 (f) System prompt length=1024

图 7：具体而言，子图 (a)–(c) 呈现了资源受限条件下的延迟结果，其中 GPU 资源在多个用户之间同时共享。相比之下，子图 (d)–(f) 说明了资源充足环境中的延迟结果。左侧 y 轴表示平均响应延迟，右侧 y 轴表示标准化延迟。

每层的注意力操作，会在内存访问和激活缓存使用方面产生更高的开销，从而增加每个令牌的推理延迟。因此，当请求速率达到 32 $reqs$ 时，纯云配置的平均延迟激增超过 4 秒，是基线延迟的四倍多，这表明在中等并发负载下存在明显的排队延迟和性能下降。

当前缀长度从512增加到1024时，如图7(b)-(c)所示，在仅计算执行模式下运行的系统在高请求率下会经历响应延迟的显着非线性增加。这种行为表明，由于无法解耦计算和内存资源，系统瓶颈逐渐从通信限制转变为计算限制。相比之下，CE-LSLM 的延迟增长保持相对稳定，表现出更强的鲁棒性。这一优势主要归功于KV缓存的复用以及模型计算和内存加载的并行协调，共同缓解了内存访问瓶颈，显着提高了推理稳定性和并发处理能力。

当GPU资源充足时，如图7(d)-(f)所示，随着请求率显着增加，两种架构都逐渐表现出饱和行为。然而，与纯云配置相比，CE-LSLM 始终实现较低的平均延迟，这表明在并发工作负载和资源压力下具有更强的延迟控制和更高的系统鲁棒性。此外，当前缀长度超过512时，平均延迟和归一化延迟都开始出现明显的波动。这种行为归因于小模型通常用于短文本或轻量级任务，其中较短的输出序列使每个令牌的延迟指标对微小的时间变化更加敏感。

总的来说，即使在饱和条件下，CE-LSLM架构也能保持有效的任务调度、边缘协作和缓存机制，而不会因网络负载增加而出现性能下降。这体现了并发延迟的强可控性。通过将部分计算卸载到边缘，显着减轻了云端负担，使系统能够支持更多数量的边缘部署的SLM实例。在吞吐量方面，在中等请求速率（例如 64 req/s）下，CE-LSLM 配置与纯云基线相比实现了大约 4-6× 高的吞吐量，从而验证了协作推理架构中边缘侧角色分配的有效性和合理性。

七．结论

在将生成式人工智能服务扩展到无线边缘网络的过程中，有限的内存和计算能力已成为LLM部署的关键瓶颈。为了应对这一挑战，提出了一种协作推理架构，将 LLM 与 SLM 集成。在6G网络背景下探索了一种新颖的服务机制，旨在共同优化生成处理和通信。所提出的架构结合了分层缓存共享和流水线计算负载并行方案，从而实现了推理执行和无线通信资源之间的协调优化。此外，在云不可用或网络连接暂时中断的情况下，边缘模型仍然可以通过以下方式执行部分推理任务：

shared cache reuse, exhibiting an initial form of distributed autonomy. Experimental results demonstrated that the proposed system significantly outperformed the conventional cloud-only baseline in terms of edge-side response latency and throughput, while maintaining comparable generation quality. These results further highlighted the system's potential in supporting privacy preservation and scalability in multi-tenant edge deployment scenarios.

In addition, this work has theoretically and empirically demonstrated the feasibility of enabling collaborative inference through cache sharing across both cloud-edge and edge-edge model deployments. Future work will focus on exploring efficient state synchronization and scheduling mechanisms among edge-side models to support more flexible and resilient distributed inference strategies.

REFERENCES

[1] M. Z. Chowdhury, M. Shahjalal, S. Ahmed, and Y. M. Jang, "6g wireless communication systems: Applications, requirements, technologies, challenges, and research directions," *IEEE Open Journal of the Communications Society*, vol. 1, pp. 957–975, 2020.

[2] N. Rajatheva, I. Atzeni, E. Bjornson, A. Bourdoux, S. Buzzi, J.-B. Dore, S. Erkucuk, M. Fuentes, K. Guan, Y. Hu *et al.*, "White paper on broadband connectivity in 6g," *arXiv preprint arXiv:2004.14247*, 2020.

[3] W. Wu, C. Zhou, M. Li, H. Wu, H. Zhou, N. Zhang, X. S. Shen, and W. Zhuang, "Ai-native network slicing for 6g networks," *IEEE Wireless Communications*, vol. 29, no. 1, pp. 96–103, 2022.

[4] X. You, C.-X. Wang, J. Huang, X. Gao, Z. Zhang, M. Wang, Y. Huang, C. Zhang, Y. Jiang, J. Wang *et al.*, "Towards 6g wireless communication networks: Vision, enabling technologies, and new paradigm shifts," *Science China Information Sciences*, vol. 64, pp. 1–74, 2021.

[5] S. Hisaharo, Y. Nishimura, and A. Takahashi, "Optimizing llm inference clusters for enhanced performance and energy efficiency," *Authorea Preprints*, 2024.

[6] J. Kundu, W. Guo, A. BanaGozar, U. De Alwis, S. Sengupta, P. Gupta, and A. Mallik, "Performance modeling and workload analysis of distributed large language model training and inference," in *2024 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2024, pp. 57–67.

[7] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 3505–3506.

[8] V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 341–353, 2023.

[9] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro *et al.*, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–15.

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.

[11] L. Zhu, X. Wang, W. Zhang, and R. W. Lau, "Relayattention for efficient large language model serving with long system prompts," *arXiv preprint arXiv:2402.14808*, 2024.

[12] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt cache: Modular attention reuse for low-latency inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 325–338, 2024.

[13] Y. Jiang, S. Wang, V. Valls, B. J. Ko, W.-H. Lee, K. K. Leung, and L. Tassiulas, "Model pruning enables efficient federated learning on edge devices," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 12, pp. 10 374–10 386, 2022.

[14] Z. Yu, Z. Wang, Y. Li, H. You, R. Gao, X. Zhou, S. R. Bommu, Y. K. Zhao, and Y. C. Lin, "Edge-llm: Enabling efficient large language model adaptation on edge devices via layerwise unified compression and adaptive layer tuning and voting," 2024. [Online]. Available: https://arxiv.org/abs/2406.15758

[15] Y. Liu, H. Du, D. Niyato, J. Kang, S. Cui, X. Shen, and P. Zhang, "Optimizing mobile-edge ai-generated everything (aigx) services by prompt engineering: Fundamental, framework, and case study," *IEEE Network*, 2023.

[16] H. Jin and Y. Wu, "Ce-collm: Efficient and adaptive large language models through cloud-edge collaboration," *arXiv preprint arXiv:2411.02829*, 2024.

[17] B. Lin, C. Zhang, T. Peng, H. Zhao, W. Xiao, M. Sun, A. Liu, Z. Zhang, L. Li, X. Qiu *et al.*, "Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache," *arXiv preprint arXiv:2401.02669*, 2024.

[18] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–16.

[19] X. Ma, G. Fang, and X. Wang, "Llm-pruner: On the structural pruning of large language models," 2023. [Online]. Available: https://arxiv.org/abs/2305.11627

[20] E. Frantar and D. Alistarh, "Sparsegpt: Massive language models can be accurately pruned in one-shot," 2023. [Online]. Available: https://arxiv.org/abs/2301.00774

[21] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan *et al.*, "Cachegen: Kv cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 38–56.

[22] Y. Yang, Z. Cao, Q. Chen, L. Qin, D. Yang, H. Zhao, and Z. Chen, "Kvsharer: Efficient inference via layer-wise dissimilar kv cache sharing," 2024. [Online]. Available: https://arxiv.org/abs/2410.18517

[23] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. K. Zhou, "Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference," 2024. [Online]. Available: https://arxiv.org/abs/2407.11550

[24] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, "Model tells you what to discard: Adaptive kv cache compression for llms," in *Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@ NeurIPS 2023)*.

[25] Y. Chen, R. Li, Z. Zhao, C. Peng, J. Wu, E. Hossain, and H. Zhang, "Netgpt: An ai-native network architecture for provisioning beyond personalized generative services," *IEEE Network*, 2024.

[26] S. Jin, X. Liu, Q. Zhang, and Z. M. Mao, "Compute or load kv cache? why not both?" *arXiv preprint arXiv:2410.03065*, 2024.

[27] Y. Zhang, P. Huang, K. Zhou, H. Wang, J. Hu, Y. Ji, and B. Cheng, "{OSCA}: An {Online-Model} based cache allocation scheme in cloud block storage systems," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 785–798.

[28] M. Klabunde, T. Schumacher, M. Strohmaier, and F. Lemmerich, "Similarity of neural network models: A survey of functional and representational measures," *ACM Computing Surveys*, Apr. 2025. [Online]. Available: http://dx.doi.org/10.1145/3728458

[29] S. Kornblith, M. Norouzi, H. Lee, and G. E. Hinton, "Similarity of neural network representations revisited," *CoRR*, vol. abs/1905.00414, 2019. [Online]. Available: http://arxiv.org/abs/1905.00414

[30] Y. Xu, Z. Jie, H. Dong, L. Wang, X. Lu, A. Zhou, A. Saha, C. Xiong, and D. Sahoo, "Think: Thinner key cache by query-driven pruning," *arXiv preprint arXiv:2407.21018*, 2024.

[31] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, vol. 1, no. 2, 2023.

[32] S. Narayan, S. B. Cohen, and M. Lapata, "Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization," *ArXiv*, vol. abs/1808.08745, 2018.

[33] D. Hendrycks, C. Burns, S. Basart, A. Critch, J. Li, D. Song, and J. Steinhardt, "Aligning ai with shared human values," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

共享缓存重用，展示了分布式自治的初始形式。实验结果表明，所提出的系统在边缘侧响应延迟和吞吐量方面显着优于传统的纯云基线，同时保持了可比较的发电质量。这些结果进一步凸显了该系统在支持多租户边缘部署场景中的隐私保护和可扩展性方面的潜力。

此外，这项工作从理论上和实证上证明了通过跨云边缘和边缘模型部署的缓存共享来实现协作推理的可行性。未来的工作将重点探索边缘侧模型之间高效的状态同步和调度机制，以支持更灵活、更有弹性的分布式推理策略。

参考

[1] M. Z. Chowdhury、M. Shahjalal、S. Ahmed 和 Y. M. Jang，“6g 无线通信系统：应用、要求、技术、挑战和研究方向”，*IEEE Open Journal of the Communications Society*，卷。1，第 957–975 页，2020 年。 [2] N. Rajatheva、I. Atzeni、E. Bjornson、A. Bourdoux、S. Buzzi、J.-B。 Dore, S. Erkucuk, M. Fuentes, K.guan, Y. Hu *et al.*，“6G 宽带连接白皮书”，*arXiv preprint arXiv:2004.14247*，2020 年。 [3] W. Wu, C. Zhou, M. Li, H. Wu, H. Zhou, N. Zhang, X. S. Shen 和 W. Zhuang，“6G 网络的 Ai-native 网络切片”，*IEEE Wireless Communications*，卷。29、没有。1，第 96-103 页，2022 年。 [4] X. You，C.-X。王，黄，X.高，Z.张，M.王，Y.黄，C.张，Y.江，J.王*et al.*，“迈向6G无线通信网络：愿景、支持技术和新范式转变，”*Science China Information Sciences*，卷。64，第 1–74 页，2021 年。[5] S. Hisaharo、Y. Nishimura 和 A. Takahashi，“优化 llm 推理集群以增强性能和能源效率”，*Authorea Preprints*，2024 年。[6] J. Kundu、W.Guo、A. BanaGozar、U. De Alwis、S. Sengupta、P. Gupta 和A. Mallik，“分布式大语言模型训练和推理的性能建模和工作负载分析”，*2024 IEEE International Symposium on Workload Characterization (IISWC)*。IEEE，2024 年，第 57-67 页。 [7] J. Rasley、S. Rajbhandari、O. Ruwase 和 Y. He，“Deepspeed：系统优化能够训练具有超过 1000 亿个参数的深度学习模型”，载于 *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*，2020 年，第 3505–3506 页。 [8] V. A. Korthikanti、J. Casper、S. Lym、L. McAfee、M. Andersch、M. Shoeybi 和 B. Catanzaro，“减少大型变压器模型中的激活重新计算”，*Proceedings of Machine Learning and Systems*，卷。5，第 341–353 页，2023 年。 [9] D. Narayanan、M. Shoeybi、J. Casper、P. LeGresley、M. Patwary、V. Korthikanti、D. Vainbrand、P. Kashinkunti、J. Bernauer、B. Catanzaro *et al.*，“使用 megatron-lm 对 GPU 集群进行高效大规模语言模型训练”，载于 *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*，2021 年，第 1-15 页。

[10] W. Kwon、Z. Li、S. Zhuang、Y. Shen、L. Cheng、C. z alez、H. Zhang 和 I. Stoica，“使用 pagedattention 服务的大语言模型的高效内存管理”，*Proceedings of the 29th Symposium on Operating Systems Principles*，2023 年，第 611-626 页。

[11] g、W. Zhang 和 R. W. Lau，“在长系统提示下实现高效大语言模型服务的 Relayattention”，*arXiv preprint arXiv:2402.14808* 2024 年。

[12] I. Gim，G. Chen，S.-s。 Lee、N. Sarda、A. Khandelwal 和 L.zhong，“提示缓存：低延迟推理的模块化注意重用”，*Proceedings of Machine Learning and Systems*，卷。6，第 325–338 页，2024 年。

[13] Y. Jiang、S. Wang、V. Valls、B. J. Ko、W.-H。 Lee、K. K. Leung 和 L. Tassiulas，“模型修剪可以在边缘设备上实现高效的联合学习”，*IEEE Transactions on Neural Networks and Learning Systems*，卷。 34、没有。 12，第 10 374–10 386 页，2022 年。

[14] Z. Yu、Z. Wang、Y. Li、H. You、R. Gau、X. Zhou、S. R. Bommu、Y. K. Zhao 和 Y. C. Lin，“Edge-llm：通过分层统一压缩和自适应层调整和投票在边缘设备上实现高效的大语言模型自适应”，2024 年。[在线]。可用：https://arxiv.org/abs/2406.15758 [15] Y. Liu、H. Du、D. Niyato、J. Kang、S. Cui、X. Shen 和 P. Zhu，“通过即时工程优化移动边缘人工智能生成的一切 (aigx) 服务：基础、框架和案例研究”，*IEEE Network*，2023 年。 [16] H. Jin 和Y. Wu，“Ce-collm：通过云边缘协作实现高效和自适应的大语言模型”，*arXiv preprint arXiv:2411.02829*，2024。 [17] B. Lin，C. Zhang，T. Peng，H. Zhu，W. Xiao，M. Sun，A. Liu，Z. Zhang，L. Li，X. Qiu *et al.*，“Infinite-llm：针对具有分散和分布式的长上下文的高效 llm 服务kvcache”，*arXiv preprint arXiv:2401.02669*，2024 年。[18] S. Rajbhandari、J. Rasley、O. Ruwase 和 Y. He，“零：训练万亿参数模型的内存优化”，见 *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*。IEEE，2020 年，第 1-16 页。 [19] X. Ma，G. Fang，X. Wang，“Llm-pruner：论大语言模型的结构剪枝”，2023。[在线]。可用：https://arxiv.org/abs/2305.11627 [20] E. Frantar 和 D. Alistarh，“Sparsegpt：可以一次性准确地修剪大规模语言模型”，2023 年。[在线]。可用：https://arxiv.org/abs/2301.00774 [21] Y. Liu、H. Li、Y. Cheng、S. Ray、Y. Huang、Q. Zhang、K. Du、J. Yao、S. Lu、G. Ananthanarayanan *et al.*，“Cachegen：用于快速大语言模型服务的 Kv 缓存压缩和流式处理”，*Proceedings of the ACM SIGCOMM 2024 Conference*，2024 年，第 177 页。 38-56。 [22] Y. Yang、Z. Cao、Q. Chen、L.qin、D. Yang、H. Zhu 和 Z. Chen，“Kvsharer：通过分层异种 kv 缓存共享进行高效推理”，2024 年。[在线]。可用：https://arxiv.org/abs/2410.18517 [23] Y. Feng、J. Lv、Y. Cao、X. Xie 和 S. K. Zhou，“Ada-kv：通过自适应预算分配优化 kv 缓存驱逐以实现高效的 llm 推理”，2024 年。[在线]。可用：https://arxiv.org/abs/2407.11550 [24] S. Ge、Y.Zhang、L.Liu、M.Zhang、J.Han 和 J.Gao，“模型告诉您要丢弃什么：llms 的自适应 kv 缓存压缩”，见 *Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@ NeurIPS 2023)*。 [25] Y. Chen、R. Li、Z. Zhu、C. Peng、J. Wu、E. Hossain 和 H. Zhu，“Netgpt：一种用于提供超越个性化生成服务的人工智能原生网络架构”，*IEEE Network*，2024。 [26] S. Jin、X. Liu、Q. Zhang 和 Z. M. Mao，“计算或加载 kv 缓存？为什么不两者都做呢？”*arXiv preprint arXiv:2410.03065*，2024 年。[27] Y. 张、P. Huang、K. Zhou、H. Wang、J. Hu、Y. Ji 和 B. Cheng，“{OSCA}：云块存储系统中基于 {在线模型}的缓存分配方案”，*2020 USENIX Annual Technical Conference (USENIX ATC 20)*，2020 年，第 785-798 页。 [28] M. Klabunde、T. Schumacher、M. Strohmaier 和 F. Lemmerich，“神经网络模型的相似性：功能性和代表性测量的调查”，*ACM Computing Surveys*，2025 年 4 月。[在线]。可用：http://dx.doi.org/10.1145/3728458 [29] S. Kornblith、M. Norouzi、H. Lee 和 G. E. Hinton，“重新审视神经网络表示的相似性”，*CoRR*，卷。绝对/1905.00414，2019。[在线]。网址：http://arxiv.org/abs/1905.00414 [30] Y. Xu、Z. Jie、H. Dong、L. Wang、X. Lu、A. Zhou、A. Saha、C. Xiong 和 D. Sahoo，“思考：通过查询驱动修剪来精简密钥缓存”，*arXiv preprint arXiv:2407.21018*，2024 年。 [31] W. X. Zhu、K. Zhou、J. Li、T.唐X.王，Y.侯，Y.敏，B.张，J.张，Z.董*et al.*，“大语言模型综述”，*arXiv preprint arXiv:2303.18223*，卷。 1、没有。 2023 年 2 月。 [32] S. Narayan、S. B. Cohen 和 M. Lapata，“不要给我详细信息，只提供摘要！用于极端摘要的主题感知卷积神经网络”，*ArXiv*，卷。 abs/1808.08745，2018。[33] D. Hendrycks、C. Burns、S. Basart、A. Critch、J. Li、D. Song 和 J. Steinhardt，“使人工智能与人类共同价值观保持一致”，*Proceedings of the International Conference on Learning Representations (ICLR)*，2021 年。

APPENDIX

In this appendix, we provide a detailed mathematical derivation of the prunable distance length for attention heads in ALiBi-based models.

A. Invariance Properties of CKA

The Centered Kernel Alignment (CKA) similarity metric exhibits several desirable invariance properties that make it particularly suitable for comparing neural network representations [29]:

1) Scale invariance: We consider the representation matrix of each layer denoted as $O_e \in \mathbb{R}^{n\times d}$, where nn is the sequence length and dd is the hidden dimension. Let

$$O_c^{(l)} = \alpha O_e^{(l)}, \quad \alpha \in \mathbb{R} \tag{21}$$

This indicates that all sample representations are scaled by a shared constant factor α, which is commonly observed in operations such as LayerNorm and residual connections.

Then, the kernel matrices can be computed as follows:

$$S_c = O_c O_c^T = \alpha^2 O_e O_e^T = \alpha^2 S_e \tag{22}$$

The following formulation can be derived by combining Eqs. 12 and 22 :

$$\begin{aligned} HSIC\left(S_e, S_c\right) &= \frac{1}{(N-1)^2} tr\left(H_N S_e H_N \alpha^2 S_e\right) \\ &= \alpha^2 \cdot HSIC\left(S_e, S_e\right) \end{aligned} \tag{23}$$

By substituting the above formulation into the normalization Eqs. 13, we have

$$\begin{aligned} m_{CKA}\left(S_e, S_c\right) &= \frac{\alpha^2 \cdot HSIC\left(S_e, S_e\right)}{\sqrt{HSIC\left(S_e, S_e\right) \cdot \alpha^4 \cdot HSIC\left(S_e, S_e\right)}} \\ &= 1 \end{aligned} \tag{24}$$

Hence, we have

$$m_{CKA}\left(O_e, \alpha O_e\right) = m_{CKA}\left(O_e, O_c\right) \tag{25}$$

2) Orthogonal invariance: Let the layer representation matrix be $O_e \in \mathbb{R}^{n\times d}$. After applying an orthogonal transformation, it becomes:

$$O_c = O_e Q \tag{26}$$

where $Q^T Q = I$.

Subsequently, the kernel matrix can be expressed as:

$$S_e = O_e O_e^T \tag{27}$$

Since orthogonal transformations do not alter the inner product structure between samples, it follows that:

$$S_c = O_c O_c^T = (O_e Q)(O_e Q)^T = O_e Q Q^T O_e^T = S \tag{28}$$

Finally, it can be verified that

$$\begin{aligned} &HSIC\left(S_e, S_c\right) = HSIC\left(S_e, S_e\right) \\ &m_{CKA}\left(S_e, S_c\right) = 1 \end{aligned} \tag{29}$$

3) Permutation invariance: Given a layer representation matrix $O_e \in \mathbb{R}^{n\times d}$, after a permutation of the feature dimensions, that is,

$$O_c = O_e P \tag{30}$$

where P is a permutation matrix.

Accordingly, the kernel matrix can be expressed as

$$S_e = O_e O_e^T \tag{31}$$

The permutation matrix satisfies $P^\top = P^{-1}$ and $PP^\top = I$, thus the inner product remains unchanged, and can be expressed as

$$S_c = O_c O_c^T = (O_e P)(O_e P)^T = O_e P P^T O_e^T = S \tag{32}$$

Thus, CKA can be defined as Taking the logarithm of both sides of the above equation yields:

$$m_{CKA}\left(S_e, S_c\right) = m_{CKA}\left(S_e, S_e\right) = 1 \tag{33}$$

附录

在本附录中，我们提供了基于 ALiBi 的模型中注意力头的可修剪距离长度的详细数学推导。

A. Invariance Properties of CKA

中心核对齐（CKA）相似性度量表现出几个理想的不变性特性，这使得它特别适合比较神经网络表示[29]：

1) Scale invariance: 我们考虑每层的表示矩阵，表示为 $O_e \in \mathbb{R}^{n\times d}$，其中nn是序列长度，dd是隐藏维度。让

$$O_c^{(l)} = \alpha O_e^{(l)}, \quad \alpha \in \mathbb{R} \tag{21}$$

这表明所有样本表示都按共享常数因子 α 进行缩放，这在 LayerNorm 和残差连接等操作中常见。

然后，核矩阵可以计算如下：

$$S_c = O_c O_c^T = \alpha^2 O_e O_e^T = \alpha^2 S_e \tag{22}$$

通过结合方程式可以得出以下公式： 12 和 22 :

$$\begin{aligned} HSIC\left(S_e, S_c\right) &= \frac{1}{\left(N-1\right)^2} tr\left(H_N S_e H_N \alpha^2 S_e\right) \\ &= \alpha^2 \cdot HSIC\left(S_e, S_e\right) \end{aligned} \tag{23}$$

通过将上述公式代入归一化方程。 13、我们有

$$\begin{aligned} m_{CKA}\left(S_e, S_c\right) &= \frac{\alpha^2 \cdot HSIC\left(S_e, S_e\right)}{\sqrt{HSIC\left(S_e, S_e\right) \cdot \alpha^4 \cdot HSIC\left(S_e, S_e\right)}} \\ &= 1 \end{aligned} \tag{24}$$

因此，我们有

$$m_{CKA}\left(O_e, \alpha O_e\right) = m_{CKA}\left(O_e, O_c\right) \tag{25}$$

2) Orthogonal invariance: 令层表示矩阵为 $O_e \in \mathbb{R}^{n\times d}$。应用正交变换后，变为：

$$O_c = O_e Q \tag{26}$$

其中$Q^T Q = I$。

随后，核矩阵可以表示为：

$$S_e = O_e O_e^T \tag{27}$$

由于正交变换不会改变样本之间的内积结构，因此：

$$S_c = O_c O_c^T = (O_e Q)(O_e Q)^T = O_e Q Q^T O_e^T = S \tag{28}$$

最后可以验证的是

$$\begin{aligned} &HSIC\left(S_e, S_c\right) = HSIC\left(S_e, S_e\right) \\ &m_{CKA}\left(S_e, S_c\right) = 1 \end{aligned} \tag{29}$$

3) Permutation invariance: 给定一个层表示矩阵 $O_e \in \mathbb{R}^{n\times d}$，在对特征维度进行排列后，即

$$O_c = O_e P \tag{30}$$

其中 P 是置换矩阵。

相应地，核矩阵可以表示为

$$S_e = O_e O_e^T \tag{31}$$

置换矩阵满足$P^\top = P^{-1}$和$PP^\top = I$，因此内积保持不变，可以表示为

$$S_c = O_c O_c^T = (O_e P)(O_e P)^T = O_e P P^T O_e^T = S \tag{32}$$

因此，CKA 可以定义为 对上式两边取对数，得到：

$$m_{CKA}\left(S_e, S_c\right) = m_{CKA}\left(S_e, S_e\right) = 1 \tag{33}$$